

Lab:-01->AIM: C programming with Assembly

Steps:-

1. Write ALP for far procedure.
2. Write main program in C.
3. Compile both separately to generate .obj files
4. Create project in turbo C and add both .obj files.
 - a. Add .obj file of main program.
 - b. Add .obj file of assembly program.
5. Run your project.

1. Find minimum of two integer numbers in an assembly function (FAR Procedure). The numbers are passed from main program in C and result is passed back from assembly function to the C program.

Ans:-

MIN.ASM:-

```
public _minimum
extrn _num1:word, _num2:word
_text segment
assume cs:_text
_minimum proc far
    mov ax, _num1
    mov bx, _num2
    cmp ax, bx
    ja skip
    ret
skip:
    mov ax, _num2
    ret
_minimum endp
int 3
_text ends
End
```

LAB11.C(C Code):-

```
#include<conio.h>
#include<stdio.h>
```

```
int num1=20, num2=25, ans;  
int extern far minimum(int,int);
```

```
void main()  
{  
    clrscr();  
    ans = minimum(num1, num2);  
    printf("Answer = %d", ans);  
    getch();  
}
```

Output:-20

2.Convert temperature given in Celsius to Fahrenheit using assembly function (FAR Procedure). The assembly function is called from main program in C.

Ans:-

CF.ASM:-

```
public _CtoF  
extrn _num:word,
```

```
_text segment  
assume cs:_text
```

```
_CtoF proc far  
    mov ax, _num  
    mov bl, 9  
    mul bl  
    mov bl, 5  
    div bl  
    add ax, 32  
    ret  
_CtoF endp
```

```
int 3  
_text ends  
End
```

LAB12.C(C Code):-

```
#include<conio.h>  
#include<stdio.h>
```

```

int num = 100, f;
int extern far CtoF(int);

void main()
{
    clrscr();
    f = CtoF(num);
    printf("Fahrenheit = %d", f);
    getch();
}

```

Output:-212

3.Calculate the LCM of two integer numbers in an assembly function (FAR Procedure). The numbers are passed from main program in C and LCM is passed back from the assembly function back to C program.

Ans:-

LCM.ASM:-

```

public _lcm
extrn _num1:word, _num2:word

```

```

_text segment
assume cs:_text

```

```

_lcm proc far
    mov ax, _num1
    mov bx, _num2
    mov cx, ax
    mov dx, bx

```

```

loop1:
    cmp ax, bx
    jz exit
    cmp ax, bx
    ja skip
    add ax, cx
    jmp loop1

```

```

skip:
    add bx, dx

```

```
    jmp loop1
```

```
exit:
```

```
    ret
```

```
_lcm endp
```

```
int 3
```

```
_text ends
```

```
End
```

LAB13.C(C Code):-

```
#include<conio.h>
```

```
#include<stdio.h>
```

```
int num1 = 20, num2 = 25, ans;
```

```
int extern far lcm(int,int);
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    ans = lcm(num1, num2);
```

```
    printf("LCM = %d", ans);
```

```
    getch();
```

```
}
```

Output:-100

LAB 2

AIM :- Introduction to 8087 programming and floating point representation.

Steps :-

1. Write ALP for the given problem.
2. Open dosbox and assemble the .asm file using TASM.
3. Generate the .exe file for the same using tlink command in TASM.
4. Debug the exe file using td <filename.exe>

1. Write an ALP to do addition of three floating point numbers using 8087 instruction set. Test case data: $x = 3.5$ $y = 5.0$ $z = 2.2$

Ans:-

DATA SEGMENT

x DD 3.5

y DD 5.0

z DD 2.2

result DD ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA

START:

MOV AX, DATA

MOV DS, AX

FLD x

FADD y

FADD z

FSTP result

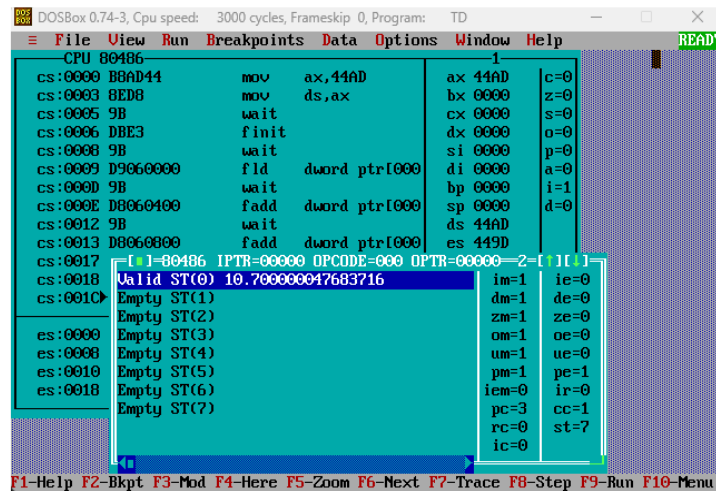
MOV AH, 4CH

INT 21H

CODE ENDS

END START

Output:-



2. Write an ALP to find area of a circle using 8087 instruction set.

Test case data: Pi = 3.1472 radius = 5.0

Ans:-

DATA SEGMENT

pi DD 3.1472

r DD 5.0

area DD ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA

START:

MOV AX, DATA

MOV DS, AX

FLD r

FMUL r

FMUL pi

FSTP area

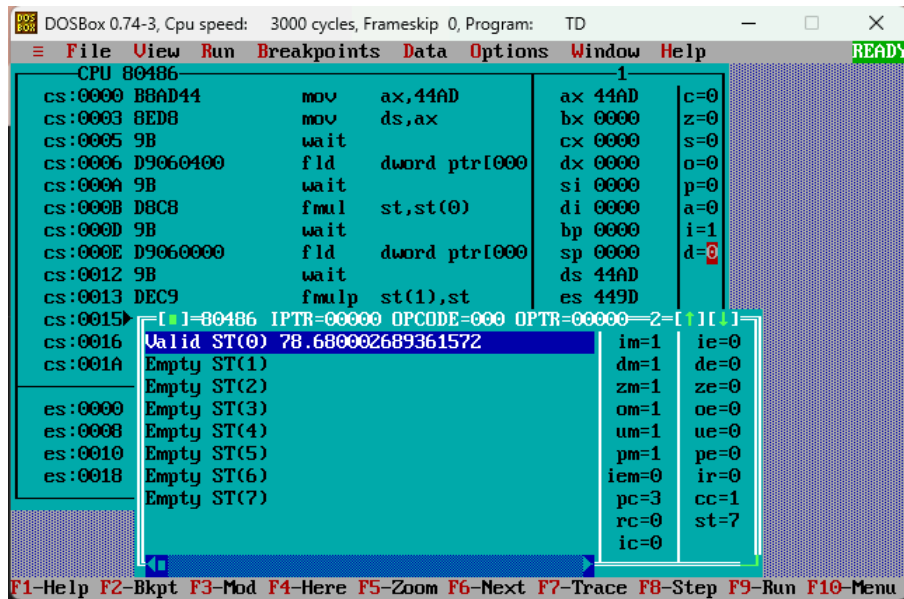
MOV AH, 4CH

INT 21H

CODE ENDS

END START

Output:-



3. Write an ALP to find volume of sphere using 8087 instruction set.

Test case data: $\pi = 3.1472$ radius = 5.0

Ans:-

DATA SEGMENT

pi DD 3.1472

r DD 5.0

four DD 4.0

three DD 3.0

vol DD ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA

START:

MOV AX, DATA

MOV DS, AX

FLD r

FMUL r

FMUL r

FMUL pi

FMUL four

FDIV three

FSTP vol

MOV AH, 4CH

INT 21H

CODE ENDS

END START

Output:-

The screenshot shows the DOSBox 0.74-3 interface. The CPU is 80486. The assembly code is as follows:

```
44AF:001C 9B          wait
44AF:001D D9060800    fld        dword ptr[000]
44AF:0021 9B          wait
44AF:0022 D8360C00    fdiv       dword ptr[000]
44AF:0026 9B          wait
44AF:0027 D8C9        fmul       st,st(1)
44AF:0029 9B          wait
44AF:002A D9161000    fst        dword ptr[001]
44AF:002E B8004C      mov        ax,4C00
44AF:0031 CD21      int        21
```

The registers are shown on the right:

ax	0192	c	=1
bx	1D09	z	=0
cx	A97A	s	=1
dx	0000	o	=0
si	0050	p	=0
di	1500	a	=0
bp	0100	i	=1
sp	0106	d	=1
ds	1D09		
es	1A01		

The stack is shown on the left:

44AF:0033	[1]-80486	IPTR=00000	OPCODE=000	OPTR=00000	2=[1][1]	
44AF:0035	Valid ST(0)	524.53335126241048			im=1	ie=0
44AF:0037	Valid ST(1)	393.40001344680786			dm=1	de=0
	Valid ST(2)	125			zm=1	ze=0
449D:0000	Empty ST(3)				om=1	oe=0
449D:0008	Empty ST(4)				um=1	ue=0
449D:0010	Empty ST(5)				pm=1	pe=1
449D:0018	Empty ST(6)				iem=0	ir=0
	Empty ST(7)				pc=3	cc=1
					rc=0	st=5
					ic=0	

The bottom of the window shows the DOSBox menu: F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu

4. Write an ALP to find $c = \sqrt{a^2 + b^2}$

Test case data: a = 5.0 b = 3.0

Ans:-

DATA SEGMENT

a DD 5.0

b DD 3.0

c DD ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA

START:


```

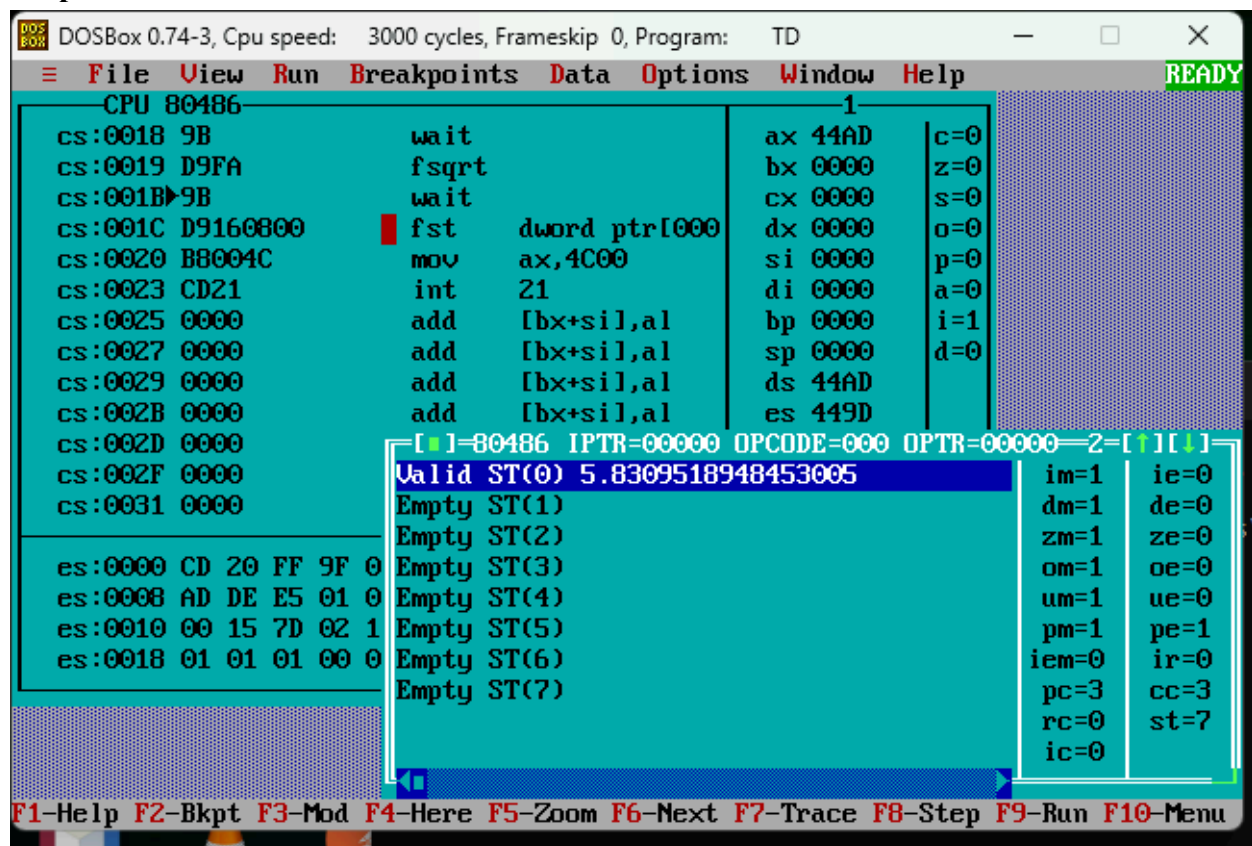
MOV AX, DATA
MOV DS, AX

FLD a
FMUL a
FLD b
FMUL b
FADD
FSQRT
FSTP c

MOV AH, 4CH
INT 21H
CODE ENDS
END START

```

Output:-



ACA LAB 3-4

**AIM: 8087 programs using Transcendental Instructions
(Trigonometric and exponentiation instructions)**

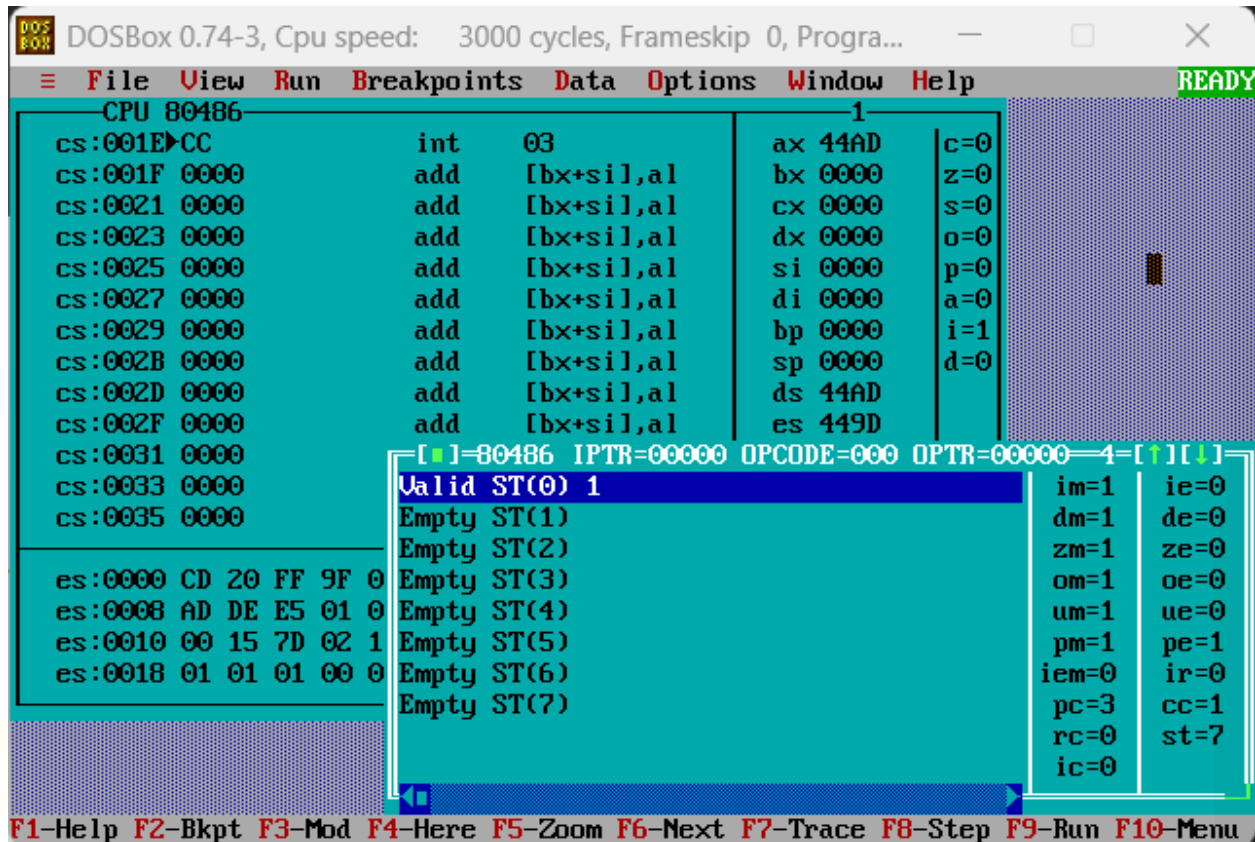
Study Trigonometric and other instructions and implement programs mentioned in the PDF.

1. Compute tan(x):-

Code:-

```
data segment
    x    dd 180.0    ; constant used for degree → radian conversion
    angle dd 45.0    ; angle in degrees
    red  dd ?        ; result storage
data ends
code segment
    assume cs:code, ds:data
start:
    mov ax, data
    mov ds, ax
    finit                ; initialize x87 FPU
    fldpi                ; ST(0) =  $\pi$ 
    fld angle            ; ST(0) = angle, ST(1) =  $\pi$ 
    fmul                 ; ST(0) = angle *  $\pi$  (ST(1) popped)
    fdiv x               ; ST(0) = (angle *  $\pi$ ) / 180
                        ; converts degrees to radians
    fptan                ; ST(0) = 1.0
                        ; ST(1) = tan(radians)
    fmul                 ; ST(0) = tan(radians) * 1.0
                        ; multiplication has no effect
    fst red              ; store tan(angle in radians) into red
    int 3                ; breakpoint (debug)
code ends
end start
```

Output:-



2. Compute sin(x):-

Code:-

data segment

```
x      dd 180.0      ; constant for degree → radian conversion
angle  dd 30.0       ; input angle in degrees
two    dd 2.0        ; constant 2.0
one    dd 1.0        ; constant 1.0
red    dd ?          ; temporary / final result
```

data ends

code segment

```
assume cs:code, ds:data
```

start:

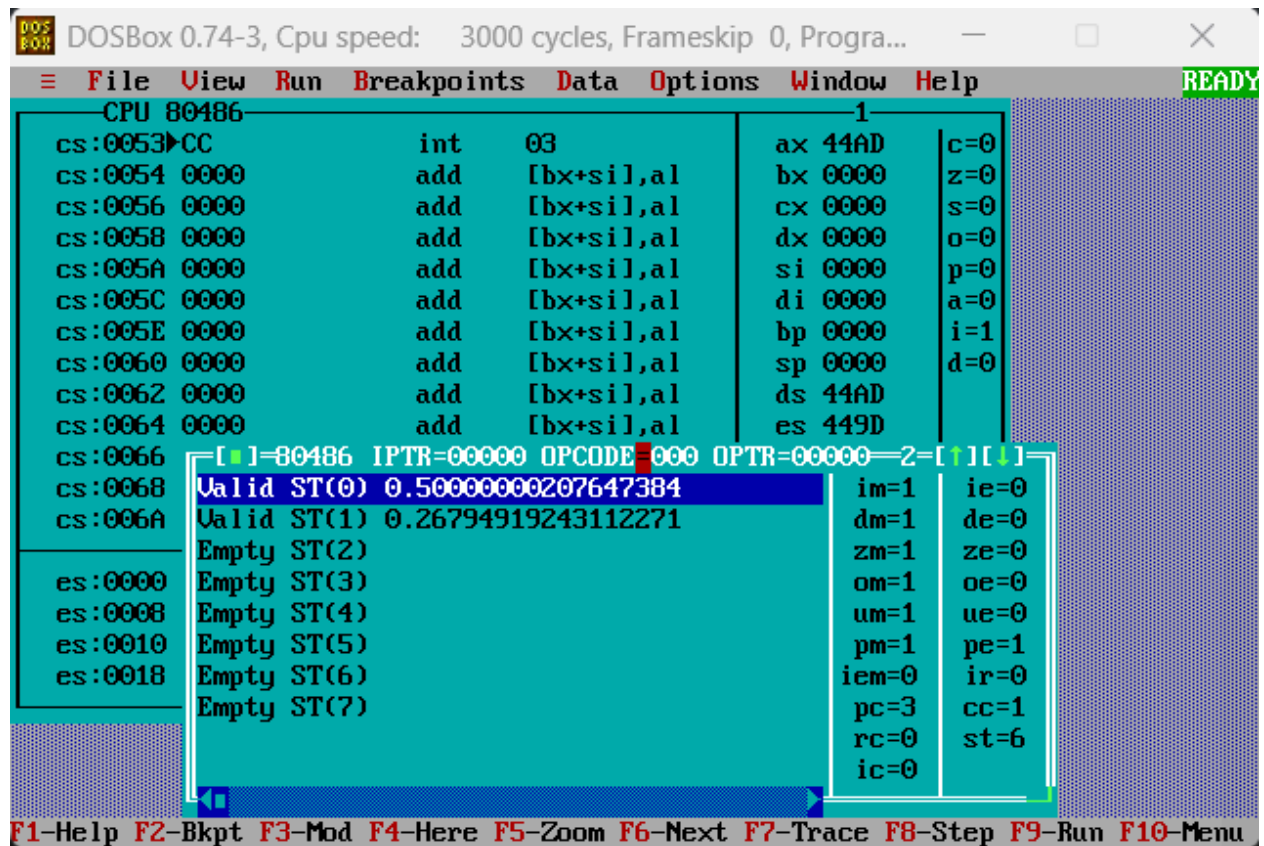
```
mov ax, data
mov ds, ax
finit      ; initialize x87 FPU
fldpi      ; ST(0) = π
fld angle  ; ST(0) = angle, ST(1) = π
fmul       ; ST(0) = angle * π
fdiv x     ; ST(0) = (angle * π) / 180 → radians
```

```

fdiv two      ; ST(0) = radians / 2
fptan         ; ST(0) = 1.0
              ; ST(1) = tan(angle / 2)
fmul          ; ST(0) = tan(angle / 2)
fst red       ; red = tan(a / 2)
fld red       ; ST(0) = tan(a/2), ST(1) = tan(a/2)
fmul          ; ST(0) = tan²(a / 2)
fld one       ; ST(0) = 1.0
fadd          ; ST(0) = 1 + tan²(a / 2)
fstp angle    ; angle = 1 + tan²(a / 2)
              ; pop ST(0)
fld red       ; ST(0) = tan(a / 2)
fmul two      ; ST(0) = 2 · tan(a / 2)
fdiv angle    ; ST(0) = 2·tan(a/2) / (1 + tan²(a/2))
fst red       ; store final result
int 3         ; breakpoint (debug)
code ends
end start

```

Output:-



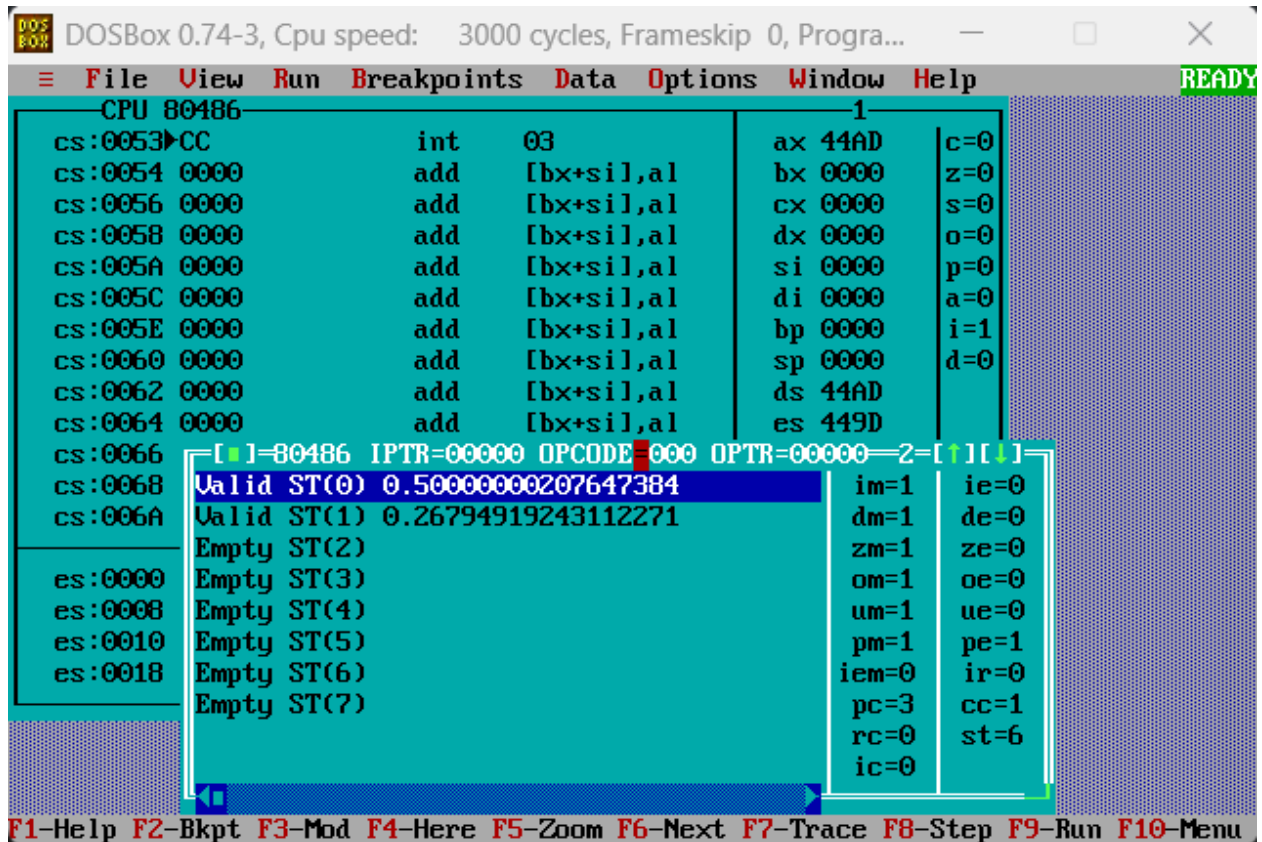
3. Compute cos(x):-

Code:-

```
data segment
    x    dd 180.0    ; constant for degree → radian conversion
    angle dd 30.0    ; input angle in degrees
    two  dd 2.0      ; constant 2.0
    one  dd 1.0      ; constant 1.0
    red  dd ?        ; result storage
data ends

code segment
    assume cs:code, ds:data
start:
    mov ax, data
    mov ds, ax
    finit                ; initialize x87 FPU
    fldpi                ; ST(0) =  $\pi$ 
    fld angle            ; ST(0) = angle, ST(1) =  $\pi$ 
    fmul                 ; ST(0) = angle *  $\pi$ 
    fdiv x               ; ST(0) = (angle *  $\pi$ ) / 180 → radians
    fdiv two             ; ST(0) = radians / 2
    fptan                ; ST(0) = 1.0
                        ; ST(1) = tan(angle / 2)
    fmul                 ; ST(0) = tan(angle / 2)
    fst red              ; red = tan(a / 2)
    fld red              ; ST(0) = tan(a/2), ST(1) = tan(a/2)
    fmul                 ; ST(0) = tan2(a / 2)
    fst red              ; red = tan2(a / 2)
    fld one              ; ST(0) = 1.0
    fadd                 ; ST(0) = 1 + tan2(a / 2)
    fstp angle           ; angle = 1 + tan2(a / 2)
                        ; pop ST(0)
    fld one              ; ST(0) = 1.0
    fsub red             ; ST(0) = 1 - tan2(a / 2)
    fdiv angle           ; ST(0) = (1 - tan2(a/2)) / (1 + tan2(a/2))
    fst red              ; store final result
    int 3                ; breakpoint (debug)
code ends
end start
```

Output:-



4. Compute x^y :-

Code:-

data segment

```

cw dw 07FFh      ; control word
y dd 3.7          ; exponent
x dd 2.9          ; base
ans dd ?          ; result storage

```

data ends

code segment

```

assume cs:code, ds:data

```

start:

```

mov ax, data
mov ds, ax
finit                ; initialize x87 FPU
fldcw cw             ; load control word
fld y                ; ST(0) = y
fld x                ; ST(0) = x, ST(1) = y
fyl2x                ; ST(0) = y * log2(x)
fst ans              ; save full value
frndint              ; ST(0) = integer part

```

```

fld ans          ; ST(0) = full value, ST(1) = integer part
fsub st(0), st(1) ; ST(0) = fractional part
f2xm1           ; ST(0) = 2^(fraction) - 1
fld1            ; ST(0) = 1.0
fadd            ; ST(0) = 2^(fraction)
fscale          ; 2^(fraction) * 2^(integer) = x^y
fst ans         ; store final result
int 3           ; breakpoint (debug)

```

code ends

end start

Output:-

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: TD

File View Run Breakpoints Data Options Window Help

CPU 80486

cs:002C 9B	wait	ax 44AD	c=1
cs:002D DEC1	faddp st(1),st	bx 0000	z=0
cs:002F 9B	wait	cx 0000	s=1
cs:0030 D9FD	fscale	dx 0000	o=0
cs:0032 9B	wait	si 0000	p=1
cs:0033 DDD9	fstp st(1)	di 0000	a=1
cs:0035 9B	wait	bp 0000	i=1
cs:0036 D9160A00	fst dword ptr[000	sp 0000	d=0
cs:003A 0000	add [bx+si],al	ds 44AD	
cs:003C 0000	add [bx+si],al	es 449D	
cs:003E	[EIP]=80486 IPTR=00000 OPCODE=000 OPTR=0000-2=[EIP]		
cs:0040	Valid ST(0) 44.441127650871897		
cs:0042	Empty ST(1)		
	Empty ST(2)		
es:0000	Empty ST(3)		
es:0008	Empty ST(4)		
es:0010	Empty ST(5)		
es:0018	Empty ST(6)		
	Empty ST(7)		

im=1 ie=0
dm=1 de=0
zm=1 ze=0
om=1 oe=0
um=1 ue=0
pm=1 pe=1
iem=1 ir=0
pc=3 cc=1
rc=1 st=7
ic=0

F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu

5. Compute xsqrt(y) + ysqrt(x):-

Code:-

```

data segment
    x    dd 4.0
    y    dd 9.0
    ans  dd ?
    temp dd ?
data ends
code segment

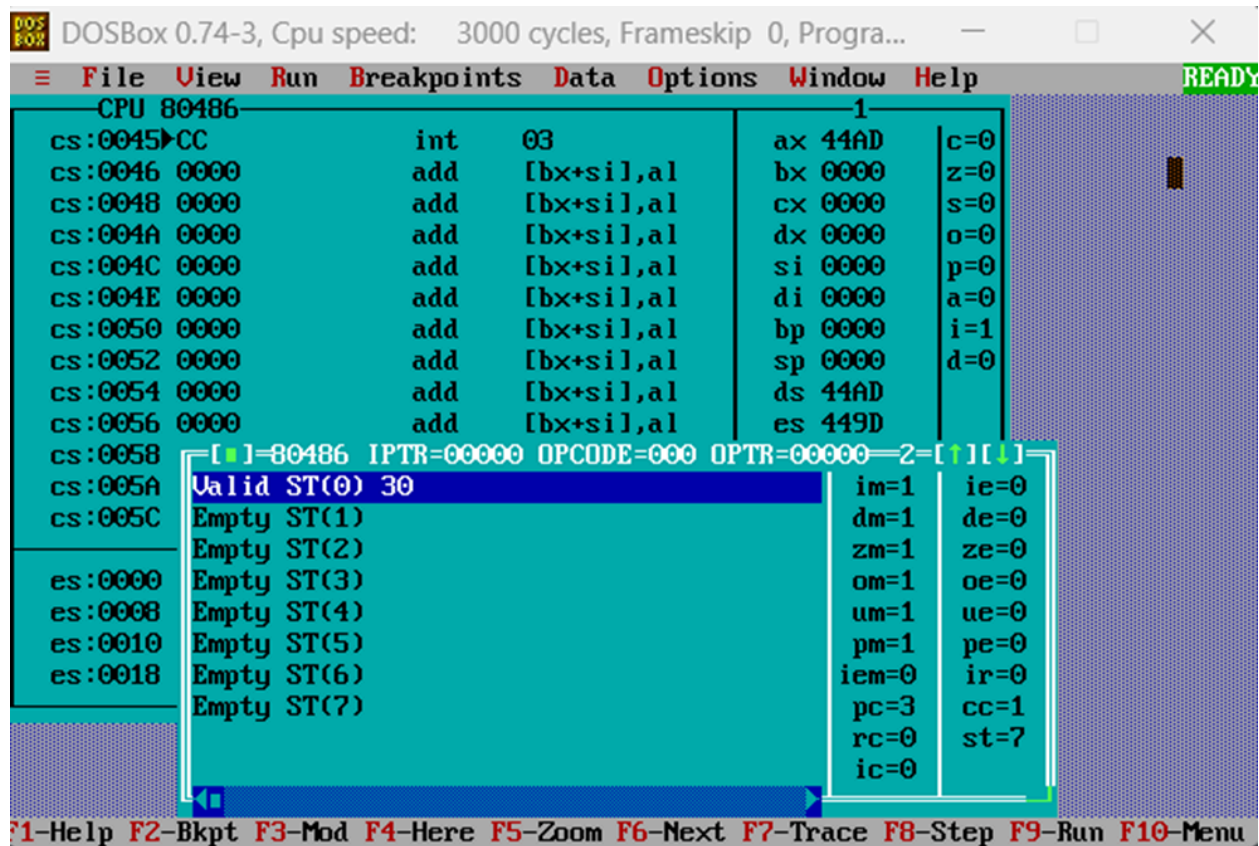
```

```

assume cs:code, ds:data
start:
    mov ax, data
    mov ds, ax
    finit          ; initialize FPU
    fld x          ; ST(0) = x
    fld y          ; ST(0) = y, ST(1) = x
    fsqrt          ; ST(0) = sqrt(y), ST(1) = x
    fmul           ; ST(0) = x * sqrt(y)
    fld y          ; ST(0) = y, ST(1) = x*sqrt(y)
    fld x          ; ST(0) = x, ST(1) = y, ST(2) = x*sqrt(y)
    fsqrt          ; ST(0) = sqrt(x), ST(1) = y, ST(2) = x*sqrt(y)
    fmul           ; ST(0) = y * sqrt(x), ST(1) = x*sqrt(y)
    fadd           ; ST(0) = x*sqrt(y) + y*sqrt(x)
    fst ans        ; store result
    int 3          ; breakpoint (debug)
code ends
end start

```

Output:-

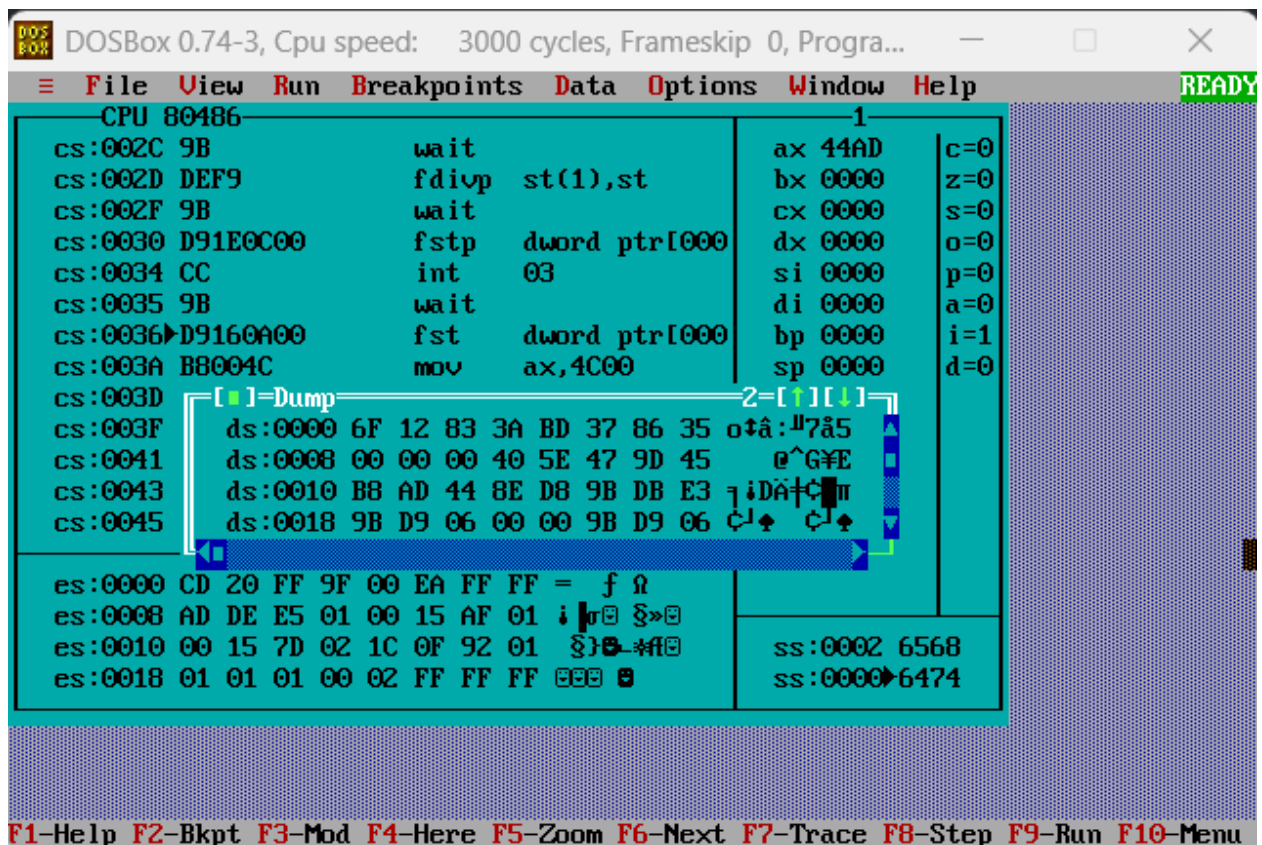


6. Find resonance frequency:-

Code:-

```
data segment
    one dd 1.0          ; constant 1.0
    two dd 2.0          ; constant 2.0
    L   dd 0.001        ; inductance (Henries)
    C   dd 0.000001     ; capacitance (Farads)
    F   dd ?            ; result (frequency)
data ends
code segment
    assume cs:code, ds:data
start:
    mov ax, data
    mov ds, ax
    finit                ; initialize x87 FPU
    fld L                ; ST(0) = L
    fmul C               ; ST(0) = L * C
    fsqrt                ; ST(0) =  $\sqrt{L * C}$ 
    fldpi                ; ST(0) =  $\pi$ , ST(1) =  $\sqrt{L * C}$ 
    fmul                 ; ST(0) =  $\pi * \sqrt{L * C}$ 
    fmul two             ; ST(0) =  $2\pi\sqrt{L * C}$ 
    fstp F               ; F =  $2\pi\sqrt{L * C}$ , pop ST(0)
    fld one              ; ST(0) = 1.0
    fdiv F               ; ST(0) =  $1 / (2\pi\sqrt{L * C})$ 
    fst F               ; store final result in F
    int 3                ; breakpoint (debug)
code ends
end start
```

Output:-



7. Compute tan inverse (y/x):-

Code:-

data segment

```
x    dd 1.0      ; input value
con  dd 2.0      ; conversion factor (radians → degrees)
red  dd ?        ; stores angle in radians
angle dd ?       ; stores angle in degrees
```

data ends

code segment

```
assume cs:code, ds:data
```

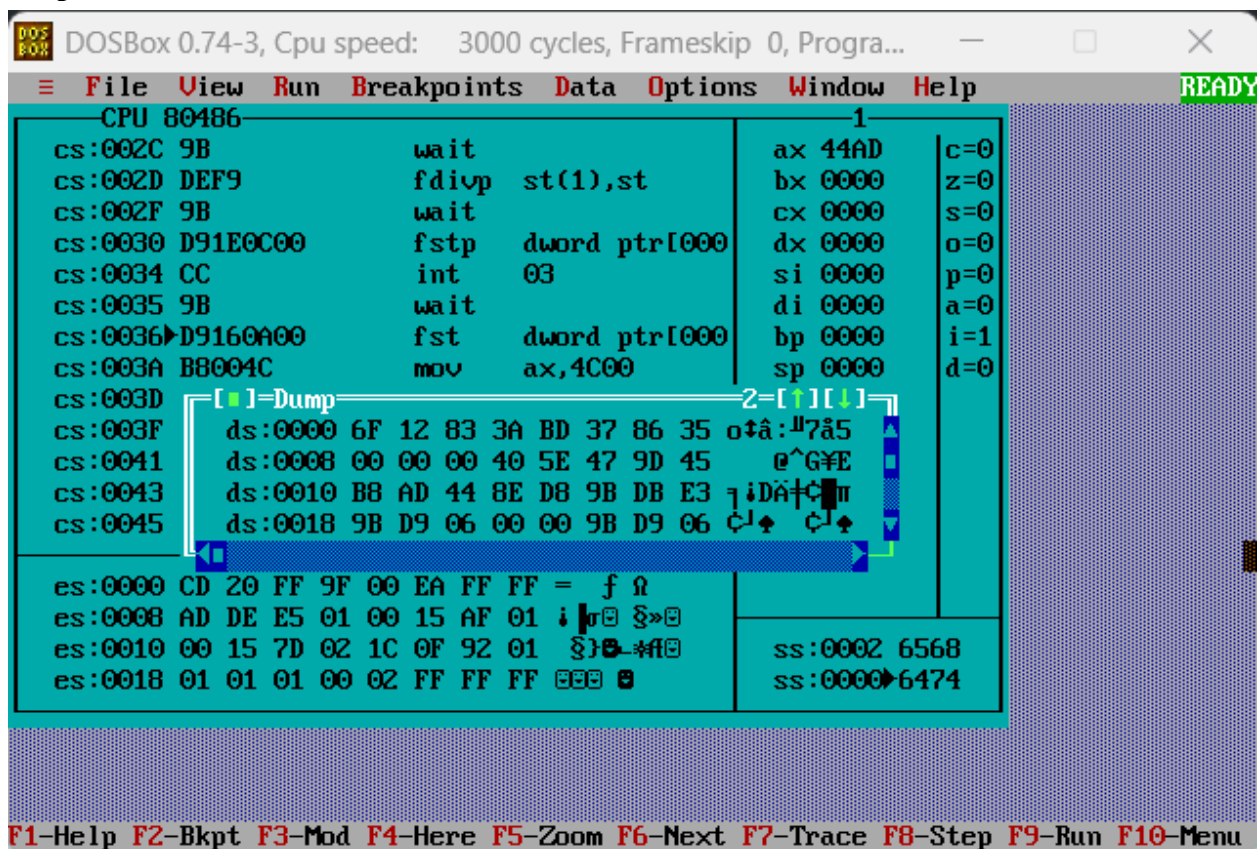
start:

```
mov ax, data
mov ds, ax
finit      ; initialize x87 FPU
fld x      ; ST(0) = x
fld1      ; ST(0) = 1.0, ST(1) = x
fpatan     ; ST(0) = arctan(ST(1) / ST(0))
           ; ST(0) = arctan(x / 1) in radians
fst red    ; red = arctan(x) (radians)
fldpi     ; ST(0) = π
```

```

fdiv          ; ST(0) = red /  $\pi$ 
fmul con      ; ST(0) = (red /  $\pi$ ) * 180
fst angle     ; angle = arctan(x) in degrees
int 3         ; breakpoint (debug)
code ends
end start
Output:-

```



8. Compute area of a cone:-

Code:-

```

data segment
    r    dd 3.0      ; radius
    h    dd 5.0      ; height
    area dd ?        ; result
data ends

code segment
    assume cs:code, ds:data
start:
    mov ax, data
    mov ds, ax
    finit            ; initialize x87 FPU

```

```

fld r          ; ST(0) = r
fldpi          ; ST(0) =  $\pi$ , ST(1) = r
fld r          ; ST(0) = r, ST(1) =  $\pi$ , ST(2) = r
fld h          ; ST(0) = h, ST(1) = r, ST(2) =  $\pi$ , ST(3) = r
fmul h         ; ST(0) = h * h
               ; (multiplies ST(0) with memory h)
fld r          ; ST(0) = r
fmul r         ; ST(0) = r * r
fadd           ; ST(0) =  $h^2 + r^2$ 
fsqrt          ; ST(0) =  $\sqrt{h^2 + r^2}$ 
fadd           ; ST(0) =  $r + \sqrt{h^2 + r^2}$ 
fmul           ; ST(0) =  $\pi \cdot (r + \sqrt{h^2 + r^2})$ 
fmul           ; ST(0) =  $\pi \cdot r \cdot (r + \sqrt{h^2 + r^2})$ 
fst area       ; store result in area
int 3          ; breakpoint (debug)

```

code ends

end start

Output:-

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...

File View Run Breakpoints Data Options Window Help

CPU 80486

cs:0056	CC	int	03	ax	44AD	c=0
cs:0057	0000	add	[bx+si],al	bx	0000	z=0
cs:0059	0000	add	[bx+si],al	cx	0000	s=0
cs:005B	0000	add	[bx+si],al	dx	0000	o=0
cs:005D	0000	add	[bx+si],al	si	0000	p=0
cs:005F	0000	add	[bx+si],al	di	0000	a=0
cs:0061	0000	add	[bx+si],al	bp	0000	i=1
cs:0063	0000	add	[bx+si],al	sp	0000	d=0

Dump

ds:0000	00 00 A0 40 00 00 40 40	ae ee
---------	-------------------------	-------

[]=80486 IPTR=00000 OPCODE=000 OPTR=00000=3= [] []

cs:006B	Valid ST(0)	83.229759144759311	im=1	ie=0
cs:006D	Valid ST(1)	54.955424984573649	dm=1	de=0
	Valid ST(2)	5.8309518948453005	zm=1	ze=0
es:0000	Empty ST(3)		om=1	oe=0
es:0008	Empty ST(4)		um=1	ue=0
es:0010	Empty ST(5)		pm=1	pe=1
es:0018	Empty ST(6)		iem=0	ir=0
	Empty ST(7)		pc=3	cc=3
			rc=0	st=5
			ic=0	

F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu

Find out roots of the Quadratic Equation $ax^2 + bx + c = 0$, where $a \neq 0$.

Consider all the cases for Delta. (Do it in Lab-4)

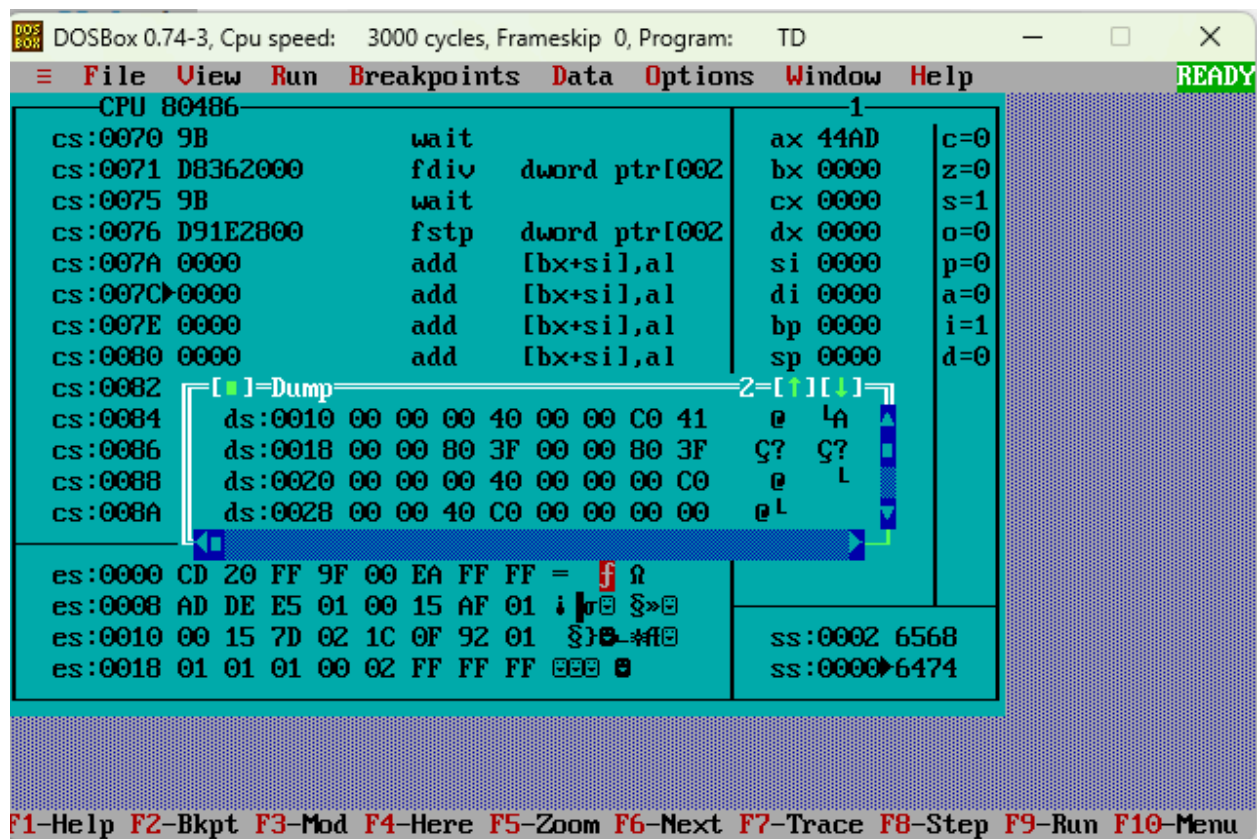
Code:-

```
; -----  
; Roots of quadratic equation  
;  $ax^2 + bx + c = 0$   
; -----  
data segment  
    a      dd 2.0  
    b      dd 4.0  
    c      dd 2.0  
    four   dd 4.0  
    two    dd 2.0  
    minus1 dd -1.0  
    root1  dd ?  
    msg    db "irrational roots$"  
data ends  
code segment  
    assume cs:code, ds:data  
start:  
    mov ax, data  
    mov ds, ax  
    finit                ; initialize FPU  
; ----- compute discriminant  $d = b^2 - 4ac$  -----  
    fld b                ; ST(0) = b  
    fmul st(0), st(0)    ; ST(0) =  $b^2$   
    fld a                ; ST(0) = a , ST(1) =  $b^2$   
    fld c                ; ST(0) = c , ST(1) = a , ST(2) =  $b^2$   
    fmulp                ; ST(0) = ac , ST(1) =  $b^2$   
    fld four             ; ST(0) = 4 , ST(1) = ac , ST(2) =  $b^2$   
    fmulp                ; ST(0) =  $4ac$  , ST(1) =  $b^2$   
    fsubp                ; ST(0) =  $d = b^2 - 4ac$   
; ----- check if  $d < 0$  -----  
    fld st(0)            ; duplicate d  
    fldz                ; ST(0) = 0 , ST(1) = d , ST(2) = d  
    fsubp                ; ST(0) = -d  
    fstsw ax  
    and ax, 0001h        ; check sign bit  
    jnz irrational       ; if  $d < 0$   
; ----- compute  $\sqrt{d}$  -----  
    fsqrt                ; ST(0) =  $\sqrt{d}$   
; ----- compute numerator  $(-b + \sqrt{d})$  -----
```

```

    fld b                ; ST(0) = b , ST(1) =  $\sqrt{d}$ 
    fmul minus1          ; ST(0) =  $-b$ 
    fadd                 ; ST(0) =  $-b + \sqrt{d}$ 
; ----- compute denominator (2a) -----
    fld a                ; ST(0) = a , ST(1) = numerator
    fld two              ; ST(0) = 2 , ST(1) = a , ST(2) = numerator
    fmulp                ; ST(0) = 2a , ST(1) = numerator
; ----- compute root -----
    fdiv                 ; ST(0) = root
    fstp root1
    jmp done
; ----- irrational roots case -----
irrational:
    mov ah, 09h
    mov dx, offset msg
    int 21h
done:
    mov ax, 4C00h
    int 21h
code ends
end start
Output:-

```



ACA Lab–5: Shared Memory

Multiprogramming–I Topic: Basics of Loop Splitting

Aim: To introduce the concepts of shared memory multiprogramming and to implement basic parallel programs using loop splitting.

Lab Exercises

1. Write header.h file and include process fork(), process join() and shared() func-tions.

```
=>
#include <sys/wait.h>
#include <unistd.h>
#include <sys/shm.h>
#include <stdlib.h>
int process_fork(int nproc){
    int j;
    for(j=1;j<nproc;j++)
        if(fork()==0) return j;
    return 0;
}
void process_join(int nproc,int id){
    int i;
    if(id==0)
        for(i=1;i<nproc;i++)
            wait(0);
    else exit(0);
}
char* shared(int size,int *shmid){
    *shmid=shmget(IPC_PRIVATE,size,0666|IPC_CREAT);
    return (shmat(*shmid,0,0));
}
```


2. Write a program to create n number of processes.

=>

```
#include "header.h"
#include <stdio.h>

int main(){
    int id,n;
    scanf("%d",&n);
    id=process_fork(n);
    printf("id = %d\n",id);
    process_join(n-1,id);
}
```

Output :

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ gcc lab52.c header.h
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
4
id = 1
id = 2
id = 0
id = 3
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$
```

3. Write a parallel program to add four variables (a, b, c, d).

=>

```
#include "header.h"
#include <stdio.h>

int main(){
    int id,a,b,c,d,*sum,shmid;
    scanf("%d %d %d %d",&a,&b,&c,&d);
    sum=(int*)shared(sizeof(int),&shmid);
    *sum=0;
    id=process_fork(4);
    if(id==0) *sum+=a;
    if(id==1) *sum+=b;
    if(id==2) *sum+=c;
    if(id==3) *sum+=d;
    process_join(3,id);
    printf("the sum = %d\n",(*sum));
}
```

Output :

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ gcc lab53.c header.h
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
1 2 3 4
the sum = 10
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ 

```

4. Write a parallel program to copy one array into another using loop splitting. =>
#include "header.h"

```

#include <stdio.h>

int main(){
    int id,a[10],*b,shmid;
    b=(int*)shared(sizeof(id)*10,&shmid);
    for(int i=0;i<10;i++){
        printf("Enter element a[%d]:",i+1);
        scanf("%d",&a[i]);
    }
    id=process_fork(2);
    if(id==0)
        for(int i=0;i<10;i+=2)
            *(b+i)=a[i];
    else
        for(int i=1;i<10;i+=2)
            *(b+i)=a[i];
    process_join(2,id);
    for(int i=0;i<10;i++)
        printf("\n b[%d]=%d",i+1,(*(b+i)));
}

```

Output :

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ gcc lab54.c header.h
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter element a[1]:1
Enter element a[2]:2
Enter element a[3]:3
Enter element a[4]:4
Enter element a[5]:5
Enter element a[6]:6
Enter element a[7]:7
Enter element a[8]:8
Enter element a[9]:9
Enter element a[10]:10

b[1]=1
b[2]=2
b[3]=3
b[4]=4
b[5]=5
b[6]=6
b[7]=7
b[8]=8
b[9]=9
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ 

```

5. Write a parallel program to copy one array into another using loop splitting and strictly using one loop.

=>

```

#include "header.h"
#include <stdio.h>
int main(){
    int id,a[10],*b,shmid;
    b=(int*)shared(sizeof(id)*10,&shmid);
    for(int i=0;i<10;i++){
        printf("Enter element a[%d]:",i+1);
        scanf("%d",&a[i]);
    }
    id=process_fork(4);
    for(int i=id;i<10;i+=4)
        *(b+i)=a[i];
    process_join(3,id);
    for(int i=0;i<10;i++)
        printf("\n b[%d]=%d",i+1,(*(b+i)));
}

```

Output :

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ gcc lab55.c header.h
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter element a[1]:1
Enter element a[2]:2
Enter element a[3]:3
Enter element a[4]:4
Enter element a[5]:5
Enter element a[6]:6
Enter element a[7]:7
Enter element a[8]:8
Enter element a[9]:9
Enter element a[10]:10

b[1]=1
b[2]=2
b[3]=3
b[4]=4
b[5]=5
b[6]=6
b[7]=7
b[8]=8
b[9]=9
b[10]=10
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

6. Write a parallel program to perform matrix addition using loop splitting.

=>

```

#include "header.h"
#include <stdio.h>

int main(){
    int id,a[3][3],b[3][3],*c,i,j,nproc=3,shmid;
    c=(int*)shared(sizeof(int),&shmid);
    for(j=0;j<3;j++)
        for(i=0;i<3;i++){
            printf("Enter element a[%d][%d]",j+1,i+1);
            scanf("%d",&a[j][i]);
        }
    for(j=0;j<3;j++)
        for(i=0;i<3;i++){
            printf("Enter element b[%d][%d]",j+1,i+1);
            scanf("%d",&b[j][i]);
        }
    id=process_fork(nproc);
    for(i=id;i<3;i++)
        for(j=0;j<3;j++)

```

```

*(c+3*i+j)=a[i][j]+b[i][j];
process_join(nproc,id);
for(j=0;j<3;j++){
for(i=0;i<3;i++)
printf("%d ",((c+i+j*3)));
printf("\n");
} }

```

Output :

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ gcc lab56.c header.h
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter element a[1][1]1
Enter element a[1][2]1
Enter element a[1][3]1
Enter element a[2][1]1
Enter element a[2][2]1
Enter element a[2][3]1
Enter element a[3][1]1
Enter element a[3][2]1
Enter element a[3][3]1
Enter element b[1][1]2
Enter element b[1][2]2
Enter element b[1][3]2
Enter element b[2][1]2
Enter element b[2][2]2
Enter element b[2][3]2
Enter element b[3][1]2
Enter element b[3][2]2
Enter element b[3][3]2
3 3 3
3 3 3
3 3 3
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

7. Write a parallel program to perform matrix multiplication using loop splitting.

=>

```

#include "header.h"
#include <stdio.h>
int main(){
int id,a[3][3],b[3][3],*c,i,k,j,nproc=3,shmid;
c=(int*)shared(sizeof(int)*3*3,&shmid);
for(j=0;j<3;j++){
for(i=0;i<3;i++){
printf("Enter element a[%d][%d] = ",j+1,i+1);
scanf("%d",&a[j][i]);

```

```

}
for(j=0;j<3;j++)
for(i=0;i<3;i++){
printf("Enter element b[%d][%d] = ",j+1,i+1);
scanf("%d",&b[j][i]);
}
id=process_fork(nproc);
for(i=id;i<3;i+=nproc)
for(j=0;j<3;j++){
*(c+3*i+j)=0;
for(k=0;k<3;k++) *(c+3*i+j)=*(c+3*i+j)+a[i][k]*b[k][j];
}
process_join(nproc,id);
for(j=0;j<3;j++){
for(i=0;i<3;i++) printf("%d ",(*(c+i+j*3)));
printf("\n");
} }

```

Output :

```

• user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ gcc lab57.c header.h
• user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter element a[1][1] = 1
Enter element a[1][2] = 0
Enter element a[1][3] = 0
Enter element a[2][1] = 0
Enter element a[2][2] = 1
Enter element a[2][3] = 0
Enter element a[3][1] = 0
Enter element a[3][2] = 0
Enter element a[3][3] = 1
Enter element b[1][1] = 1
Enter element b[1][2] = 2
Enter element b[1][3] = 3
Enter element b[2][1] = 4
Enter element b[2][2] = 5
Enter element b[2][3] = 6
Enter element b[3][1] = 7
Enter element b[3][2] = 8
Enter element b[3][3] = 9
1 2 3
4 5 6
7 8 9
• user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ █

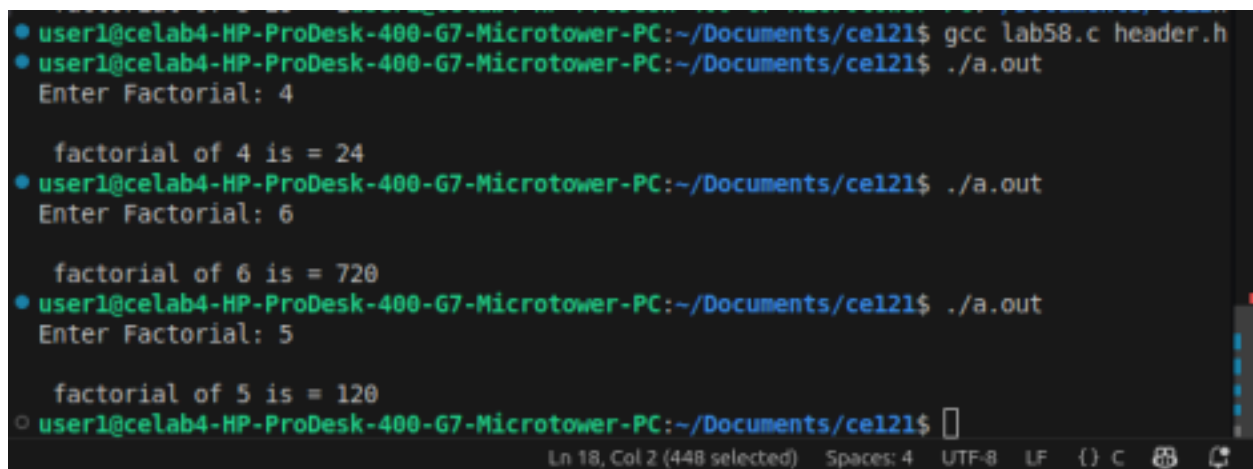
```

8. Write a parallel program to find the factorial of a number using loop splitting.

=>

```
#include "header.h"
#include <stdio.h>
int main()
{
    int id,*fact,i,n,nproc=3;
    fact=(int*)shared(sizeof(int)*nproc,&i);
    printf("Enter Factorial: ");
    scanf("%d",&n);
    id=process_fork(nproc);
    for(i=id;i<nproc;i+=nproc)
        *(fact+id)=1;
    for(i=id+1;i<=n;i+=nproc)
        *(fact+id)*=i;
    process_join(nproc,id);
    for(i=1;i<nproc;i++)
        *fact *=(fact+i);
    printf("\n factorial of %d is = %d \n",n,*fact);
}
```

Output :



```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ gcc lab58.c header.h
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter Factorial: 4

    factorial of 4 is = 24
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter Factorial: 6

    factorial of 6 is = 720
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter Factorial: 5

    factorial of 5 is = 120
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$
```

ACA Lab–6-7: Shared Memory Multiprogramming–II

Topic: Self-Scheduling

AIM: Programs using self-scheduling and locks in parallel processing.

EXERCISES:-

1. Complete Programs 5 to 8 of Loop Splitting from Previous Lab (Lab-5), which includes Matrix Addition, Multiplication, and Factorial of the given number.

=> Done in Lab 5.

2. In the header.h file created in LAB 5, include the following functions:

- **spin lock init()**
- **spin lock()**
- **spin unlock()**

=>

```
#include <sys/wait.h>
#include <unistd.h>
#include <sys/shm.h>
#include <stdlib.h>
#include <sys/sem.h>
```

```
/* Create multiple processes */
```

```
int process_fork(int nproc) {
    int j;
    for (j = 1; j < nproc; j++) {
        if (fork() == 0)
            return j;
    }
    return 0;
}
```

```
/* Join processes */
```

```
void process_join(int nproc, int id) {
    int i;
    if (id == 0) {
        for (i = 1; i < nproc; i++)
            wait(0);
    } else {
        exit(0);
    }
}
```



```

/* Shared memory creation */
char* shared(int size, int *shmid) {
    *shmid = shmget(IPC_PRIVATE, size, 0666 | IPC_CREAT);
    return (char *) shmat(*shmid, 0, 0);
}

/* Initialize spin lock */
void spin_lock_init(int *lock, int *condition) {
    int control;

    *lock = semget(IPC_PRIVATE, 1, 0666 | IPC_CREAT);

    if (*condition == 1)
        control = 0;
    else
        control = 1;

    semctl(*lock, 0, SETVAL, control);
}

/* Acquire spin lock */
void spin_lock(int *lock) {
    struct sembuf operations;

    operations.sem_num = 0;
    operations.sem_op = -1;
    operations.sem_flg = 0;

    semop(*lock, &operations, 1);
}

/* Release spin lock */
void spin_unlock(int *lock) {
    struct sembuf operations;

    operations.sem_num = 0;
    operations.sem_op = 1;
    operations.sem_flg = 0;

    semop(*lock, &operations, 1);
}

```

3. Write a program to add a constant to an array using self-scheduling.

=>

```
#include <stdio.h>
#include "header.h"

int main() {
    int *a, *next_index;
    int i, id;
    int k = 5, nproc = 100, shmid, n = 10000;

    /* Shared variable for next index */
    next_index = (int *) shared(sizeof(int), &shmid);
    *next_index = 0;

    /* Shared array */
    a = (int *) shared(sizeof(int) * n, &shmid);
    for (i = 0; i < n; i++)
        *(a + i) = 0;

    /* Create processes */
    id = process_fork(nproc);

    /* Critical section (without lock) */
    while (1) {
        i = *next_index;
        *next_index = *next_index + 1;

        if (i < n)
            *(a + i) = *(a + i) + k;
        else
            break;
    }

    /* Join processes */
    process_join(nproc, id);
}
```

Output:-



4. Write a parallel program to implement Exercise 3 using locking.

=>

```
#include <stdio.h>
#include "header.h"
```

```
int main() {
    int *a, *next_index, *lock;
    int i, id;
    int k = 5, nproc = 100, shmid, n = 10000;
    int locked = 1, unlocked = 0;

    /* Shared variable for index */
    next_index = (int *) shared(sizeof(int), &shmid);
    *next_index = 0;

    /* Shared array */
    a = (int *) shared(sizeof(int) * n, &shmid);
    for (i = 0; i < n; i++)
        *(a + i) = 0;

    /* Shared spin lock */
}
```

```

lock = (int *) shared(sizeof(int), &shmid);
spin_lock_init(lock, &unlocked);

/* Create processes */
id = process_fork(nproc);

/* Critical section protected by spin lock */
while (1) {
    spin_lock(lock);

    i = *next_index;
    *next_index = *next_index + 1;

    spin_unlock(lock);

    if (i < n)
        *(a + i) = *(a + i) + k;
    else
        break;
}

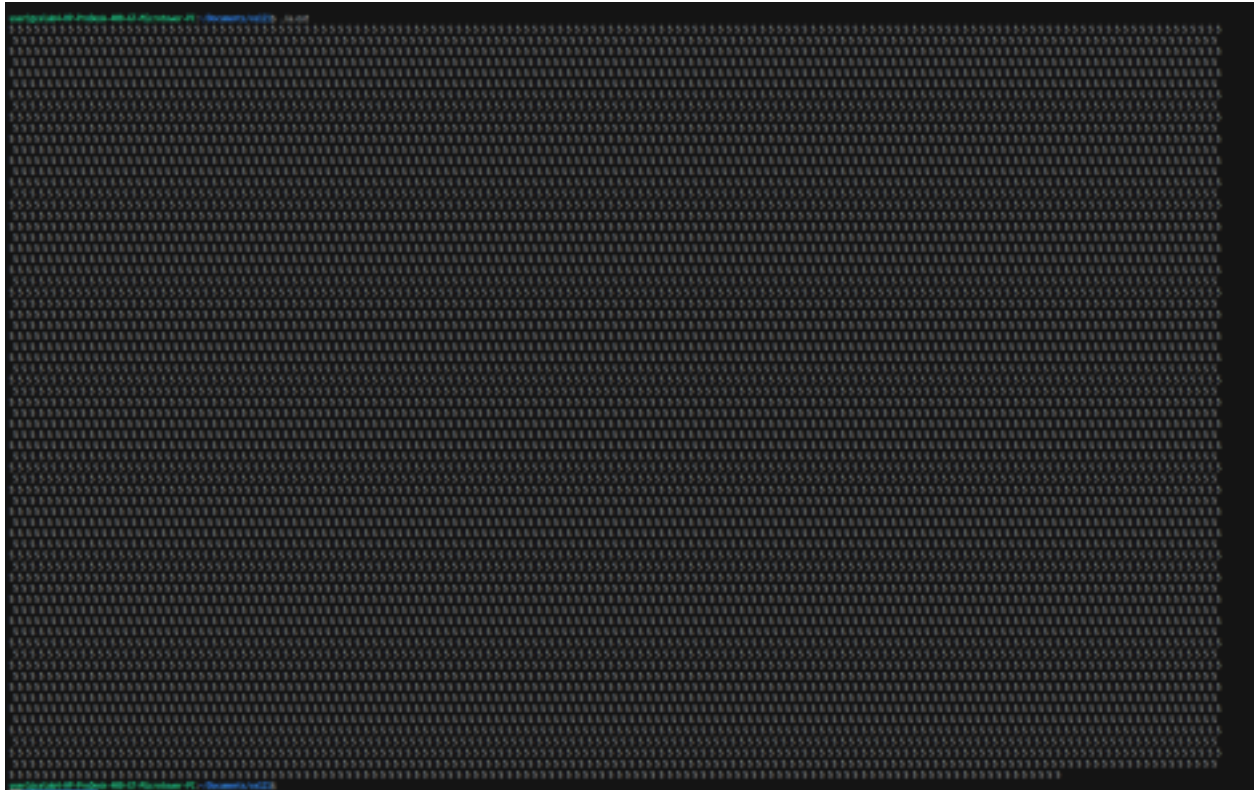
/* Join processes */
process_join(nproc, id);

/* Print result */
for (i = 0; i < n; i++)
    printf("%d ", *(a + i));

return 0;
}

```

Output:-



5. Write a parallel program to calculate the sum of numbers from 1 to n using m processes.

=>

```
#include <stdio.h>
```

```
#include "header.h"
```

```
int main() {
```

```
    int sum = 0, *final_sum;
```

```
    int i, id, k = 5, nproc, shmid, N;
```

```
    int unlocked = 0;
```

```
    int *lock1, *lock2, *index;
```

```
    /* Shared final sum */
```

```
    final_sum = (int *) shared(sizeof(int), &i);
```

```
    *final_sum = 0;
```

```
    /* Shared locks and index */
```

```
    lock1 = (int *) shared(sizeof(int), &shmid);
```

```

lock2 = (int *) shared(sizeof(int), &shmid);
index = (int *) shared(sizeof(int), &shmid);
*index = 1;
printf("Enter number : ");
scanf("%d", &N);
printf("Enter number of Processes : ");
scanf("%d", &nproc);
/* Initialize locks */
spin_lock_init(lock1, &unlocked);
spin_lock_init(lock2, &unlocked);
/* Create processes */
id = process_fork(nproc);
/* Each process computes partial sum */
while (1) {
    spin_lock(lock1);
    i = *index;
    *index = *index + 1;
    spin_unlock(lock1);
    if (i <= N)
        sum = sum + i;
    else
        break;
}
/* Update final sum safely */
spin_lock(lock2);
*final_sum = sum + *final_sum;

```

```

spin_unlock(lock2);

/* Join processes */

process_join(nproc, id);

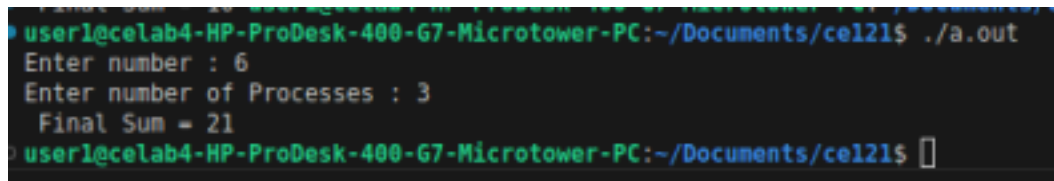
printf("Final Sum: %d\n", *final_sum);

return 0;

}

```

Output:-



```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter number : 6
Enter number of Processes : 3
Final Sum = 21
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ 

```

6. Write a parallel program to calculate the sum of an array of n elements using self-scheduling.

=>

```

#include <stdio.h>

#include "header.h"

int main() {

    int sum = 0, *final_sum;

    int id, a[10], i = 0, nproc = 4;

    int unlocked = 0;

    int *lock1, *lock2;

    int shmid, *index;

    /* Shared variables */

    final_sum = (int *) shared(sizeof(int), &shmid);

    *final_sum = 0;

    lock1 = (int *) shared(sizeof(int), &shmid);

    lock2 = (int *) shared(sizeof(int), &shmid);

    index = (int *) shared(sizeof(int), &shmid);

```

```

*index = 0;

/* Input array */
for (i = 0; i < 10; i++) {
    printf("Enter a[%d] values : ", i + 1);
    scanf("%d", &a[i]);
}

/* Initialize locks */
spin_lock_init(&lock1, &unlocked);
spin_lock_init(&lock2, &unlocked);

/* Create processes */
id = process_fork(nproc);

/* Parallel sum calculation */
while (1) {
    spin_lock(&lock1);
    i = *index;
    *index = *index + 1;
    spin_unlock(&lock1);
    if (i < 10)
        sum = sum + a[i];
    else
        break;
}

/* Combine partial sums safely */
spin_lock(&lock2);
*final_sum = sum + *final_sum;
spin_unlock(&lock2);

/* Join processes */

```



```

process_join(nproc, id);

/* Output */

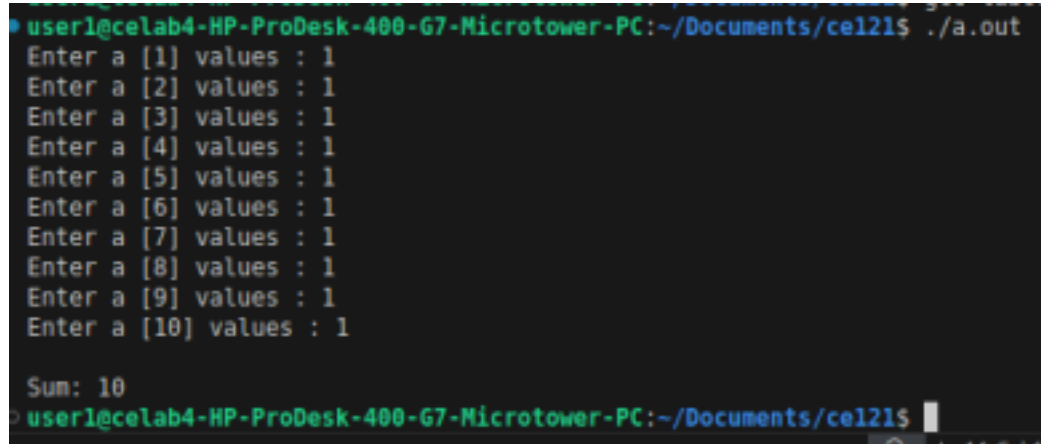
printf("\nSum: %d\n", *final_sum);

return 0;

}

```

Output:-



```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter a [1] values : 1
Enter a [2] values : 1
Enter a [3] values : 1
Enter a [4] values : 1
Enter a [5] values : 1
Enter a [6] values : 1
Enter a [7] values : 1
Enter a [8] values : 1
Enter a [9] values : 1
Enter a [10] values : 1

Sun: 10
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

7. Implementation of histogram in different ways:

(a) Create a histogram using self-scheduling.

=>

```

#include "header.h"
#include <stdlib.h>
#include <stdio.h>

#define arrSize 15

int main() {
    int a[arrSize], i;
    int *index, *histogram;
    int NoOfBins = 5, binsize;
    int *lock1, *lock2;
    int unlocked = 0, locked = 1;
    int shmidindex, shmidlock1, shmidlock2, shmidhist;
    int id, nproc, bin, amin, amax;

    /* Input array */
    for (i = 0; i < arrSize; i++) {
        printf("Enter a[%d] : ", i);
        scanf("%d", &a[i]);
    }
}

```

```
}
```

```
/* Find min and max */
```

```
amin = a[0];
```

```
amax = a[0];
```

```
for (i = 1; i < arrSize; i++) {
```

```
    if (amin > a[i])
```

```
        amin = a[i];
```

```
    if (amax < a[i])
```

```
        amax = a[i];
```

```
}
```

```
/* Calculate bin size */
```

```
binsize = (amax - amin) / NoOfBins;
```

```
/* Shared memory allocation */
```

```
index = (int *) shared(sizeof(int), &shmidindex);
```

```
lock1 = (int *) shared(sizeof(int), &shmidlock1);
```

```
lock2 = (int *) shared(sizeof(int), &shmidlock2);
```

```
histogram = (int *) shared(sizeof(int) * NoOfBins, &shmidhist);
```

```
printf("\nBin Size : %d", binsize);
```

```
printf("\nNo. Of Bins : %d\n", NoOfBins);
```

```
/* Initialize locks */
```

```
spin_lock_init(lock1, &unlocked);
```

```
spin_lock_init(lock2, &unlocked);
```

```
*index = 0;
```

```
/* Initialize histogram */
```

```
for (i = 0; i < NoOfBins; i++)
```

```
    *(histogram + i) = 0;
```

```
/* Create processes */
```

```
nproc = NoOfBins;
```

```
id = process_fork(nproc);
```

```
/* Histogram calculation */
```

```
while (1) {
```

```

    spin_lock(lock1);

    i = *index;
    *index = *index + 1;

    spin_unlock(lock1);

    if (i >= arrSize)
        break;

    bin = abs((a[i] - amin) / binsize);
    if (bin >= NoOfBins)
        bin = NoOfBins - 1;

    printf("Number %d is : %d\tBin : %d\n", i, a[i], bin);

    spin_lock(lock2);
    *(histogram + bin) += 1;
    spin_unlock(lock2);
}

/* Join processes */
process_join(nproc, id);

/* Print histogram */
for (i = 0; i < NoOfBins; i++)
    printf("No of Items in Bin (%d): %d\n", i, *(histogram + i));

return 0;
}

```

Output:-

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
enter a [0] :0
enter a [1] :11
enter a [2] :12
enter a [3] :19
enter a [4] :22
enter a [5] :23
enter a [6] :24
enter a [7] :25
enter a [8] :27
enter a [9] :29
enter a [10] :35
enter a [11] :36
enter a [12] :37
enter a [13] :45
enter a [14] :50

Bin Size : 10
No. Of Bins : 5
Number 0 is : 0 Bin : 0
Number 1 is : 11 Bin : 1
Number 2 is : 12 Bin : 1
Number 3 is : 19 Bin : 1
Number 5 is : 23 Bin : 2
Number 6 is : 24 Bin : 2
Number 7 is : 25 Bin : 2
Number 8 is : 27 Bin : 2
Number 9 is : 29 Bin : 2
Number 10 is : 35 Bin : 3
Number 4 is : 22 Bin : 2
Number 11 is : 36 Bin : 3
Number 12 is : 37 Bin : 3
Number 13 is : 45 Bin : 4
Number 14 is : 50 Bin : 4
No of Items in Bin (0): 1
No of Items in Bin (1): 3
No of Items in Bin (2): 6
No of Items in Bin (3): 3
No of Items in Bin (4): 2
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$
```

(b) Implement a histogram using loop splitting.

=>

```
#include <stdio.h>
```

```
#include "header.h"
```

```
#define arrSize 15
```

```
int main() {
    int a[arrSize];
    int NoOfBins = 5, binsize;
    int *histogram;
```

```

int *lock, unlocked = 0, locked = 1;
int shmidlock, shmidhist;
int id, nproc, bin, amin, amax, i;

/* Input array */
for (i = 0; i < arrSize; i++) {
    printf("Enter a[%d] : ", i);
    scanf("%d", &a[i]);
}

/* Find min and max */
amin = a[0];
amax = a[0];

for (i = 1; i < arrSize; i++) {
    if (amin > a[i])
        amin = a[i];
    if (amax < a[i])
        amax = a[i];
}

/* Calculate bin size */
if ((amax - amin) == 0)
    binsize = 1;
else
    binsize = (amax - amin) / NoOfBins;

/* Shared memory allocation */
lock = (int *) shared(sizeof(int) * NoOfBins, &shmidlock);
histogram = (int *) shared(sizeof(int) * NoOfBins, &shmidhist);

printf("\nBin Size : %d", binsize);
printf("\nNo. Of Bins : %d\n", NoOfBins);

/* Initialize locks (one lock per bin) */
for (i = 0; i < NoOfBins; i++)
    spin_lock_init(lock + i, &unlocked);

/* Initialize histogram */
for (i = 0; i < NoOfBins; i++)
    *(histogram + i) = 0;

```

```

printf("\n");

/* Create processes */
nproc = NoOfBins;
id = process_fork(nproc);

/* Parallel histogram computation */
for (i = id; i < arrSize; i = i + nproc) {
    bin = abs(a[i] - amin) / binsize;
    if (bin >= NoOfBins)
        bin = NoOfBins - 1;

    printf("Process(%d): Number[%d] = %d \t Bin: %d\n",
        id, i, a[i], bin);

    spin_lock(lock + bin);
    *(histogram + bin) += 1;
    spin_unlock(lock + bin);
}

printf("\n");

/* Join processes */
process_join(NoOfBins, id);

/* Print histogram */
for (i = 0; i < NoOfBins; i++)
    printf("No Of Items in Bin (%d) is %d\n",
        i, *(histogram + i));

return 0;
}

```

Output:-

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel215$ ./a.out
enter a[0] : 0
enter a[1] : 11
enter a[2] : 12
enter a[3] : 19
enter a[4] : 22
enter a[5] : 23
enter a[6] : 24
enter a[7] : 25
enter a[8] : 27
enter a[9] : 29
enter a[10] : 35
enter a[11] : 36
enter a[12] : 37
enter a[13] : 45
enter a[14] : 50

Bin Size : 10
No. Of Bins : 5

Process( 1 ):Number [1] is 11      Bin: 1
Process( 1 ):Number [6] is 24      Bin: 2
Process( 1 ):Number [11] is 36     Bin: 3

Process( 2 ):Number [2] is 12      Bin: 1
Process( 2 ):Number [7] is 25      Bin: 2
Process( 2 ):Number [12] is 37     Bin: 3

Process( 3 ):Number [3] is 19      Bin: 1
Process( 0 ):Number [0] is 0       Bin: 0
Process( 3 ):Number [8] is 27      Bin: 2
Process( 3 ):Number [13] is 45     Bin: 4
Process( 0 ):Number [5] is 23      Bin: 2
Process( 0 ):Number [10] is 35     Bin: 3

Process( 4 ):Number [4] is 22      Bin: 2
Process( 4 ):Number [9] is 29      Bin: 2
Process( 4 ):Number [14] is 50     Bin: 4

No Of Items in Bin (0) is 1
No Of Items in Bin (1) is 3
No Of Items in Bin (2) is 6
No Of Items in Bin (3) is 3
No Of Items in Bin (4) is 2
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel215$

```

(c) Implement a histogram using partial histograms.

=>

```

#include <stdio.h>
#include <stdlib.h>
#include "header.h"

```

```

#define arrSize 15

```

```

int main() {
    int a[arrSize];
    int NoOfBins, binsize;
    int *histogram;

```

```

int *lock, unlocked = 0, locked = 1;
int shmidlock, shmidhist;
int id, nproc, bin, amin, amax, i;

/* Input number of bins */
printf("Enter No Of Bins : ");
scanf("%d", &NoOfBins);

int parhist[NoOfBins];

/* Input array */
for (i = 0; i < arrSize; i++) {
    printf("Enter a[%d] : ", i);
    scanf("%d", &a[i]);
}

/* Find min and max */
amin = a[0];
amax = a[0];

for (i = 1; i < arrSize; i++) {
    if (amin > a[i])
        amin = a[i];
    if (amax < a[i])
        amax = a[i];
}

/* Calculate bin size */
if ((amax - amin) > 0)
    binsize = (amax - amin) / NoOfBins;
else
    binsize = 1;

/* Shared memory allocation */
lock = (int *) shared(sizeof(int) * NoOfBins,
&shmidlock);
histogram = (int *) shared(sizeof(int) *
NoOfBins, &shmidhist);

printf("\nBin Size : %d", binsize);
printf("\nNo. Of Bins : %d\n", NoOfBins);

```



```

/* Initialize locks */
for (i = 0; i < NoOfBins; i++)
    spin_lock_init(lock + i, &unlocked);

/* Initialize global histogram */
for (i = 0; i < NoOfBins; i++)
    *(histogram + i) = 0;

printf("\n");

/* Create processes */
nproc = NoOfBins;
id = process_fork(nproc);

/* Initialize private histogram */
for (i = 0; i < NoOfBins; i++)
    parhist[i] = 0;

/* Each process computes its private histogram */
for (i = id; i < arrSize; i = i + nproc) {
    bin = abs(a[i] - amin) / binsize;
    if (bin >= NoOfBins)
        bin = NoOfBins - 1;

    printf("Process(%d): Number[%d] = %d \t\n",
        id, i, a[i], bin);

    parhist[bin] += 1;
}

/* Merge private histograms into global histogram */
for (i = 0; i < NoOfBins; i++) {
    spin_lock(lock + i);
    *(histogram + i) += parhist[i];
    spin_unlock(lock + i);
}

printf("\n");

/* Join processes */

```

```

process_join(nproc, id);

/* Print final histogram */
for (i = 0; i < NoOfBins; i++)
    printf("No Of Items in Bin (%d) is %d\n",
           i, *(histogram + i));

return 0;
}

```

Output:-

```

user1@celab5-Veriton-M200-P500:~/Downloads/cel2932_cn/RS-232$ ./a.out
Enter No Of Bins : 5
enter a[0] : 0
enter a[1] : 11
enter a[2] : 12
enter a[3] : 19
enter a[4] : 22
enter a[5] : 23
enter a[6] : 24
enter a[7] : 25
enter a[8] : 27
enter a[9] : 29
enter a[10] : 35
enter a[11] : 36
enter a[12] : 37
enter a[13] : 45
enter a[14] : 50

Bin Size : 10
No. Of Bins : 5

Process(1):Number[1] is 11      Bin: 1
Process(1):Number[6] is 24      Bin: 2
Process(1):Number[11] is 36     Bin: 3

Process(2):Number[2] is 12      Bin: 1
Process(2):Number[7] is 25      Bin: 2
Process(2):Number[12] is 37     Bin: 3

Process(3):Number[3] is 19      Bin: 1
Process(3):Number[8] is 27      Bin: 2
Process(3):Number[13] is 45     Bin: 4

Process(0):Number[0] is 0       Bin: 0
Process(0):Number[5] is 23      Bin: 2
Process(0):Number[10] is 35     Bin: 3

Process(4):Number[4] is 22      Bin: 2
Process(4):Number[9] is 29      Bin: 2
Process(4):Number[14] is 50     Bin: 4

No Of Items in Bin (0) is 1
No Of Items in Bin (1) is 3
No Of Items in Bin (2) is 6
No Of Items in Bin (3) is 3
No Of Items in Bin (4) is 2
user1@celab5-Veriton-M200-P500:~/Downloads/cel2932_cn/RS-232$ █

```

ACA Lab-8: Shared Memory Multiprogramming-III: Barriers

AIM: Programs using barrier

EXERCISES:

1. The header.h file created in LAB 5 includes barrier_init() and barrier() function.

=>

```
#include <sys/wait.h>
```

```
#include <unistd.h>
```

```
#include <sys/shm.h>
```

```
#include <stdlib.h>
```

```
#include <sys/sem.h>
```

```
#include <stdio.h>
```

```
/* ----- Process Fork ----- */
```

```
int process_fork(int nproc) {
```

```
    int j;
```

```
    for (j = 1; j < nproc; j++) {
```

```
        if (fork() == 0)
```

```
            return j;
```

```
    }
```

```
    return 0;
```

```
}
```

```
/* ----- Process Join ----- */
```

```
void process_join(int nproc, int id) {
```

```
    int i;
```

```
    if (id == 0) {
```

```
        for (i = 1; i < nproc; i++)
```

```
            wait(0);
```

```
    } else {
```

```
        exit(0);
```

```
}
```

```

}

/* ----- Shared Memory ----- */

char* shared(int size, int *shmid) {
    *shmid = shmget(IPC_PRIVATE, size, 0666 | IPC_CREAT);
    return (char*) shmat(*shmid, 0, 0);
}

/* ----- Spin Lock ----- */

void spin_lock_init(int *lock, int *condition) {
    int control;

    *lock = semget(IPC_PRIVATE, 1, 0666 | IPC_CREAT);

    if (*condition == 1)
        control = 0;
    else
        control = 1;

    semctl(*lock, 0, SETVAL, control);
}

void spin_lock(int *lock) {
    struct sembuf operations;

    operations.sem_num = 0;
    operations.sem_op = -1;
    operations.sem_flg = 0;

    semop(*lock, &operations, 1);
}

void spin_unlock(int *lock) {
    struct sembuf operations;

    operations.sem_num = 0;

```

```

    operations.sem_op = 1;
    operations.sem_flg = 0;
    semop(*lock, &operations, 1);
}

/* ----- Barrier ----- */

void barrier_init(int *bar1, int bnum) {
    int c = 0;
    *(bar1 + 0) = bnum; // total processes
    *(bar1 + 1) = 0;    // arrival count
    *(bar1 + 2) = 0;    // release counter

    spin_lock_init((bar1 + 3), &c);
}

void barrier(int *bar) {
    int incr = 0;

    while (1) {

        spin_lock((bar + 3));

        if (incr == 0 && *(bar + 2) > 0) {
            spin_unlock((bar + 3));
            continue;
        }

        if (incr == 0) {
            *(bar + 1) = *(bar + 1) + 1;
            incr = 1;
        }

        if (*(bar + 1) < *(bar + 0) && (*(bar + 2) == 0)) {
            spin_unlock((bar + 3));
            continue;
        }
        else {

```

```

    if (*(bar + 2) == 0) {
        *(bar + 2) = *(bar + 0) - 1;
        *(bar + 1) = 0;

        spin_unlock((bar + 3));
        return;
    }
    else {
        *(bar + 2) = *(bar + 2) - 1;

        spin_unlock((bar + 3));
        return;
    }
}

```

```

spin_unlock((bar + 3));
printf("\nError in barrier");
return;
}

```

2. WAP to Calculate standard deviation with help of barrier

=>

```

#include "header.h"
#include <math.h>

```

```

int main() {

```

```

    int shmid, id, i, locked = 1, unlocked = 0;
    int *lock, nproc, *bar;
    float *average, *deviation, sum = 0.0, a[20];

```

```

    /* ----- Shared Memory Allocation ----- */

```

```

    lock    = (int *) shared(sizeof(int), &shmid);
    deviation = (float *) shared(sizeof(float), &shmid);
    bar     = (int *) shared(sizeof(int) * 4, &shmid);
    average  = (float *) shared(sizeof(float), &shmid);

```

```

*average = 0.0;
*deviation = 0.0;

/* ----- Input ----- */

printf("Enter no. of Process: ");
scanf("%d", &nproc);

/* ----- Initialize Array ----- */

for (i = 0; i < 20; i++)
    a[i] = i;

/* ----- Initialize Lock & Barrier ----- */

spin_lock_init(&lock, &unlocked);
barrier_init(&bar, nproc);

printf("\n");

/* ----- Fork Processes ----- */

id = process_fork(nproc);

/* ----- Compute Average ----- */

for (i = id; i < 20; i += nproc)
    sum = sum + a[i];

spin_lock(&lock);    // Critical Section (Average)
*average = *average + sum / 20;
spin_unlock(&lock);

barrier(&bar);    // Synchronization Barrier

/* ----- Compute Deviation ----- */

sum = 0.0;

```

```

for (i = id; i < 20; i += nproc)
    sum = sum + fabs(a[i] - *average);

spin_lock(lock);    // Critical Section (Deviation)
*deviation = *deviation + sum / 20;
spin_unlock(lock);

/* ----- Join Processes ----- */

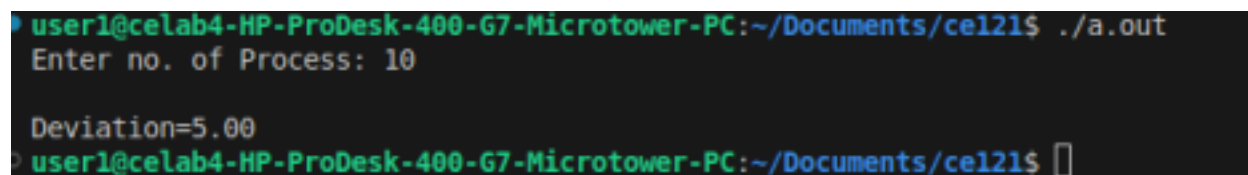
process_join(nproc, id);

printf("Deviation = %.2f\n", *deviation);

return 0;
}

```

Output :



```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Enter no. of Process: 10

Deviation=5.00
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ 

```

3. Write a parallel program to implement Histogram with a barrier.

=>

```

#include "header.h"
#include <stdio.h>

#define arrSize 15

/* ----- Find Max & Min ----- */

void find_max_min(int a[], int id, int *sm, int *sn,
                  int *lock1, int *bar, int nproc) {

    int i, amax, amin;

    if (id == 0) {
        *sm = a[0];
        *sn = a[0];
    }
}

```



```

barrier(bar);

amax = a[0];
amin = a[0];

for (i = id; i < 10; i += nproc) {
    if (amax < a[i])
        amax = a[i];

    if (amin > a[i])
        amin = a[i];
}

spin_lock(lock1);

if (*sm < amax)
    *sm = amax;

if (*sn > amin)
    *sn = amin;

spin_unlock(lock1);

barrier(bar);
}

/* ----- Main Function ----- */

int main() {

    int a[arrSize];
    int NoOfBins = 5, binsize;
    int *histogram;
    int *lock1, *lock2;
    int unlocked = 0, locked = 1;
    int *bar;

    int shmidbar, shmidlock1, shmidlock2;
    int shmidhist, shmidmax, shmidmin;

    int id, nproc, bin;
    int *amin, *amax;

```

```

int i;

/* ----- Shared Memory Allocation ----- */

amax = (int *) shared(sizeof(int), &shmidmax);
amin = (int *) shared(sizeof(int), &shmidmin);

bar = (int *) shared(sizeof(int) * 4, &shmidbar);

lock1 = (int *) shared(sizeof(int), &shmidlock1);
lock2 = (int *) shared(sizeof(int) * NoOfBins, &shmidlock2);

/* ----- Input Array ----- */

for (i = 0; i < arrSize; i++) {
    printf("Enter a[%d] : ", i);
    scanf("%d", &a[i]);
}

/* ----- Process Count ----- */

nproc = NoOfBins;

/* ----- Initialize Barrier & Locks ----- */

barrier_init(bar, nproc);

for (i = 0; i < NoOfBins; i++)
    spin_lock_init((lock2 + i), &unlocked);

spin_lock_init(lock1, &unlocked);

/* ----- Fork Processes ----- */

id = process_fork(nproc);

/* ----- Find Global Max & Min ----- */

find_max_min(a, id, amax, amin, lock1, bar, nproc);

binsize = (*amax - *amin) / NoOfBins;

histogram = (int *) shared(sizeof(int) * NoOfBins, &shmidhist);

```

```

printf("\nBin Size : %d", binsize);
printf("\nNo. Of Bins : %d\n", NoOfBins);

for (i = 0; i < NoOfBins; i++)
    *(histogram + i) = 0;

printf("\n");

/* ----- Histogram Calculation ----- */

for (i = id; i < arrSize; i += nproc) {

    bin = abs(a[i] - *amin) / binsize;

    if (bin >= NoOfBins)
        bin = NoOfBins - 1;

    printf("Process (%d): Number [%d] is %d \t Bin: %d\n",
        id, i, a[i], bin);

    spin_lock(lock2 + bin);
    *(histogram + bin) += 1;
    spin_unlock(lock2 + bin);
}

printf("\n");

/* ----- Join Processes ----- */

process_join(NoOfBins, id);

/* ----- Print Histogram ----- */

for (i = 0; i < NoOfBins; i++)
    printf("No Of Items in Bin (%d) is %d\n",
        i, *(histogram + i));

return 0;
}

```

Output :

```
user1@celab5-Veriton-M200-P500:~/Downloads/cel2932_cn/RS-232$ ./a.  
Enter No OF Bins : 5  
enter a[0] : 0  
enter a[1] : 11  
enter a[2] : 12  
enter a[3] : 19  
enter a[4] : 22  
enter a[5] : 23  
enter a[6] : 24  
enter a[7] : 25  
enter a[8] : 27  
enter a[9] : 29  
enter a[10] : 35  
enter a[11] : 36  
enter a[12] : 37  
enter a[13] : 45  
enter a[14] : 50  
  
Bin Size : 10  
No. Of Bins : 5  
  
Process(1):Number[1] is 11      Bin: 1  
Process(1):Number[6] is 24      Bin: 2  
Process(1):Number[11] is 36     Bin: 3  
  
Process(2):Number[2] is 12      Bin: 1  
Process(2):Number[7] is 25      Bin: 2  
Process(2):Number[12] is 37     Bin: 3  
  
Process(3):Number[3] is 19      Bin: 1  
Process(3):Number[8] is 27      Bin: 2  
Process(3):Number[13] is 45     Bin: 4  
  
Process(0):Number[0] is 0       Bin: 0  
Process(0):Number[5] is 23      Bin: 2  
Process(0):Number[10] is 35     Bin: 3  
  
Process(4):Number[4] is 22      Bin: 2  
Process(4):Number[9] is 29      Bin: 2  
Process(4):Number[14] is 50     Bin: 4  
  
No Of Items in Bin (0) is 1  
No Of Items in Bin (1) is 3  
No Of Items in Bin (2) is 6  
No Of Items in Bin (3) is 3  
No Of Items in Bin (4) is 2  
user1@celab5-Veriton-M200-P500:~/Downloads/cel2932_cn/RS-232$
```

ACA Lab-9+10: OpenMP I+II

AIM: To study the core features of openMP

1. Creating threads
2. Problems encountered without synchronization constructs
3. Synchronization overhead and False sharing

OpenMP Tutorial & Exercises

Lab-9

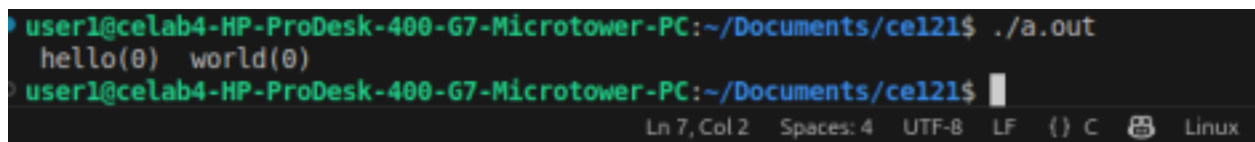
1. Study the tutorial slides 1 – 45 and perform the exercises given

a. Exercise 1A : Hello World Program

=>

```
#include <stdio.h>
#include <omp.h>
int main()
{
    int ID = 0;
    printf("hello(%d) ", ID);
    printf("world(%d)\n", ID);
    return 0;
}
```

Output :

A terminal window screenshot showing the execution of a program. The prompt is 'user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21\$'. The command './a.out' has been executed, resulting in the output 'hello(0) world(0)'. The prompt is now 'user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21\$'. The terminal status bar at the bottom shows 'Ln 7, Col 2', 'Spaces: 4', 'UTF-8', 'LF', '({} C', and 'Linux'.

b. Exercise 1B : Write a multithreaded program that prints “Hello World”

=>

```
#include <stdio.h>
#include <omp.h>
void pooh(int ID)
{
    printf("hello(%d) ", ID);
    printf("world(%d)\n", ID);
}
```

```

int main()
{
    omp_set_num_threads(50);

    #pragma omp parallel
    {
        int ID = omp_get_thread_num();
        pooh(ID);
    }

    printf("all done\n");

    return 0;
}

```

Output:-

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ gcc -fopenmp lab 91.c
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
hello(0) world(0)
hello(1) hello(3) hello(7) world(7)
world(3)
hello(17) world(17)
hello(14) hello(2) world(14)
hello(28) hello(27) hello(23) world(23)
world(27)
hello(21) world(21)
hello(20) hello(16) hello(13) hello(11) world(16)
hello(10) hello(25) world(2)
world(25)
hello(19) world(19)
hello(46) world(46)
hello(40) world(40)
hello(42) world(42)

```

Lab-10

EXERCISES:

1. Create a Sequential version of the pi program. Display the value of pi and the run time of the program. (For calculating run time use openMP library routine)

=>

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
static long num_steps = 100000000;
```

```

int main()
{
    double step, x, pi, sum = 0.0;

    step = 1.0 / (double)num_steps;

    double start = omp_get_wtime();

    for (long i = 0; i < num_steps; i++)
    {
        x = (i + 0.5) * step;
        sum += 4.0 / (1.0 + x * x);
    }

    pi = step * sum;

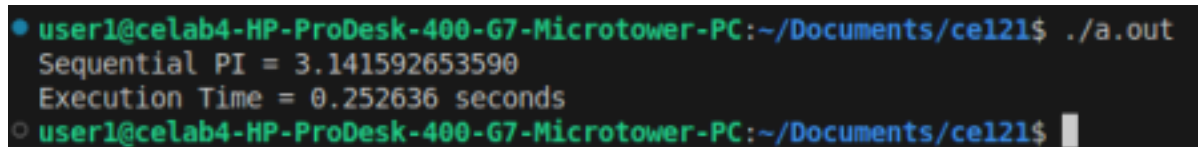
    double end = omp_get_wtime();

    printf("Sequential PI = %.12f\n", pi);
    printf("Execution Time = %f seconds\n", end - start);

    return 0;
}

```

Output:-



```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Sequential PI = 3.141592653590
Execution Time = 0.252636 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

2. Create a parallel version of the pi program using parallel construct. Display the value of pi and the run time and respective number of threads. (Run four instances of the program for number of threads ranging from 1 to 4). Use following runtime library routines. a. int omp_get_num_threads();

b. int omp_get_thread_num();

c. double omp_get_wtime();

=>

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
static long num_steps = 100000;
```

```
double step;
```

```

#define NUM_THREADS 1

int main()
{
    int i, nthreads;
    double pi, sum[NUM_THREADS];
    double start_time, end_time;

    step = 1.0 / (double)num_steps;

    omp_set_num_threads(NUM_THREADS);

    start_time = omp_get_wtime();

    #pragma omp parallel
    {
        int i, id, nthrds;
        double x;

        id = omp_get_thread_num();
        nthrds = omp_get_num_threads();

        if (id == 0)
            nthreads = nthrds;

        sum[id] = 0.0;

        for (i = id; i < num_steps; i += nthrds)
        {
            x = (i + 0.5) * step;
            sum[id] += 4.0 / (1.0 + x * x);
        }

        printf("Thread %d finished\n", id);
    }
}

```



```

for (i = 0, pi = 0.0; i < nthreads; i++)
    pi += sum[i] * step;

end_time = omp_get_wtime();

printf("Value of PI = %lf\n", pi);
printf("Number of threads = %d\n", nthreads);
printf("Execution Time = %lf seconds\n", end_time - start_time);

return 0;
}

```

Output:-

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Thread 0 finished
Value of PI = 3.141593
Number of threads = 1
Execution Time = 0.002206 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ 

```

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Thread 0 finished
Thread 1 finished
Value of PI = 3.141593
Number of threads = 2
Execution Time = 0.002554 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ 

```

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Thread 2 finished
Thread 0 finished
Thread 1 finished
Value of PI = 3.141593
Number of threads = 3
Execution Time = 0.005105 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ 

```

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Thread 0 finished
Thread 1 finished
Thread 3 finished
Thread 2 finished
Value of PI = 3.141593
Number of threads = 4
Execution Time = 0.004772 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ 

```

3. Modify your “pi program” from exercise 2 to avoid false sharing using

padding.

=>

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
static long num_steps = 100000;
```

```
double step;
```

```
#define PAD 8
```

```
#define NUM_THREADS 1
```

```
int main()
```

```
{
```

```
    int i, nthreads;
```

```
    double pi, start_time, end_time;
```

```
    double sum[NUM_THREADS][PAD];
```

```
    step = 1.0 / (double)num_steps;
```

```
    for (i = 0; i < NUM_THREADS; i++)
```

```
        sum[i][0] = 0.0;
```

```
    omp_set_num_threads(NUM_THREADS);
```

```
    start_time = omp_get_wtime();
```

```

#pragma omp parallel
{
    int id, i, nthrds;

    double x;

    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();

    if (id == 0)
        nthreads = nthrds;

    for (i = id; i < num_steps; i += nthrds)
    {
        x = (i + 0.5) * step;

        sum[id][0] += 4.0 / (1.0 + x * x);
    }
}

for (i = 0, pi = 0.0; i < nthreads; i++)
    pi += sum[i][0] * step;

end_time = omp_get_wtime();

printf("Value of PI = %lf\n", pi);

```

```

printf("Threads used = %d\n", nthreads);

printf("Execution time = %lf seconds\n", end_time - start_time);

return 0;
}

```

Output:-

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Value of PI = 3.141593
Threads used = 1
Execution time = 0.000346 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Value of PI = 3.141593
Threads used = 2
Execution time = 0.002773 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Value of PI = 3.141593
Threads used = 3
Execution time = 0.002272 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Value of PI = 3.141593
Threads used = 4
Execution time = 0.002235 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$

```

4. Modify your “pi program” from exercise 2 to avoid false sharing using a critical section.

=>

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
static long num_steps = 100000;
```

```
double step;
```

```
#define NUM_THREADS 1

int main()
{
    int nthreads;

    double pi = 0.0;

    step = 1.0 / (double)num_steps;

    omp_set_num_threads(NUM_THREADS);

    double start_time = omp_get_wtime();

    #pragma omp parallel
    {
        int i, id, nthrds;

        double x, sum;

        id = omp_get_thread_num();

        nthrds = omp_get_num_threads();

        if (id == 0)
```

```
    nthreads = nthrds;
```

```
    sum = 0.0;
```

```
    for (i = id; i < num_steps; i += nthrds)
```

```
    {
```

```
        x = (i + 0.5) * step;
```

```
        sum += 4.0 / (1.0 + x * x);
```

```
    }
```

```
    #pragma omp critical
```

```
    {
```

```
        pi += sum * step;
```

```
    }
```

```
}
```

```
double end_time = omp_get_wtime();
```

```
printf("Value of PI = %lf\n", pi);
```

```
printf("Threads used = %d\n", nthreads);
```

```
printf("Execution Time = %lf seconds\n", end_time - start_time);
```

```
return 0;
```

```
}
```

Output:-

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Value of PI = 3.141593
Threads used = 1
Execution Time = 0.001266 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$
```

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Value of PI = 3.141593
Threads used = 2
Execution Time = 0.000914 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$
```

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Value of PI = 3.141593
Threads used = 3
Execution Time = 0.000725 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$
```

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Value of PI = 3.141593
Threads used = 4
Execution Time = 0.000883 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$
```

5. Modify your “pi program” from exercise 2 to avoid false sharing using an atomic section.

=>

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
static long num_steps = 100000;
```

```
double step;
```

```
#define NUM_THREADS 1
```

```
int main()
```

```
{
```

```
    int nthreads;
```

```
    double pi = 0.0;
```

```
step = 1.0 / (double)num_steps;

omp_set_num_threads(NUM_THREADS);


double start_time = omp_get_wtime();


#pragma omp parallel

{

    int i, id, nthrds;

    double x, sum;


    id = omp_get_thread_num();

    nthrds = omp_get_num_threads();


    if (id == 0)

        nthreads = nthrds;


    sum = 0.0;


    for (i = id; i < num_steps; i += nthrds)

    {

        x = (i + 0.5) * step;
```



```
        sum += 4.0 / (1.0 + x * x);

    }

    sum = sum * step;

    #pragma omp atomic

    pi += sum;

}

double end_time = omp_get_wtime();

printf("Value of PI = %lf\n", pi);

printf("Threads used = %d\n", nthreads);

printf("Execution Time = %lf seconds\n", end_time - start_time);

return 0;
```

Lab-11: OpenMP-III : Parallel Loops, Loop sharing constructs, Reduction and Synchronization

AIM: To study the core features of openMP

- 1. Parallel Loops (Worksharing)**
- 2. Loop Worksharing constructs , the schedule clause**
- 3. Reduction**
- 4. Synchronization: Barrier, Master Construct and Single Construct.**

EXERCISES:

1. Create a parallel version of the pi program using loop construct. Display the value of pi and the run time and respective number of threads. (Run four instances of the program for number of threads ranging from 1 to 4). Use following runtime library routines.

a. int omp_get_num_threads();

b. int omp_get_thread_num();

c. double omp_get_wtime();

=>

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
static long num_steps = 100000;
```

```
double step;
```

```
int main()
```

```
{
```

```
    for (int k = 1; k <= 4; k++)
```

```
    {
```

```
        int i, nthreads;
```

```
        double pi = 0.0, sum = 0.0;
```

```
        double start_time, end_time;
```

```
        step = 1.0 / (double) num_steps;
```

```
        omp_set_num_threads(k);
```

```
        start_time = omp_get_wtime();
```

```
        #pragma omp parallel
```

```
        {
```

```
            int id = omp_get_thread_num();
```

```
            int nthrds = omp_get_num_threads();
```

```

#pragma omp single
{
    if (id == 0)
        nthreads = nthrds;
}

double x;

#pragma omp for private(x) reduction(+:sum)
for (i = 0; i < num_steps; i++)
{
    x = (i + 0.5) * step;
    sum += 4.0 / (1.0 + x * x);
}

pi = sum * step;
end_time = omp_get_wtime();

printf("Value of PI = %lf\n", pi);
printf("Number of threads = %d\n", nthreads);
printf("Execution Time = %lf seconds\n\n", end_time - start_time);
}

return 0;
}

```

Output:-

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel121$ ./a.out
Value of PI = 3.141593
Number of threads = 1
Execution Time = 0.001031 seconds

Value of PI = 3.141593
Number of threads = 2
Execution Time = 0.000741 seconds

Value of PI = 3.141593
Number of threads = 2
Execution Time = 0.000662 seconds

Value of PI = 3.141593
Number of threads = 2
Execution Time = 0.000421 seconds

```

2. Write an OpenMP program to perform matrix multiplication using loop construct.

=>

```

#include <stdio.h>
#include <omp.h>

#define M 3
#define N 3
#define P 3

int main()
{
    int A[M][N], B[N][P], C[M][P];
    int i, j, k;

    /* Initialize matrix A */
    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            A[i][j] = i + j;

    /* Initialize matrix B */
    for (i = 0; i < N; i++)
        for (j = 0; j < P; j++)
            B[i][j] = i * j;

    /* Initialize result matrix C */
    for (i = 0; i < M; i++)
        for (j = 0; j < P; j++)
            C[i][j] = 0;

    double start_time = omp_get_wtime();

    #pragma omp parallel for private(j, k) shared(A, B, C)
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < P; j++)
        {
            for (k = 0; k < N; k++)
            {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    double end_time = omp_get_wtime();

    printf("Execution Time: %f seconds\n", end_time - start_time);
}

```

```

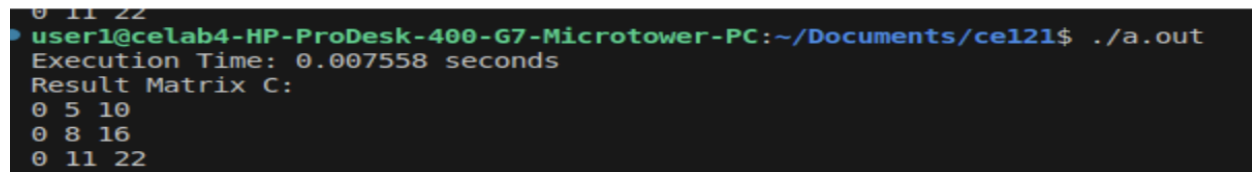
printf("Result Matrix C:\n");
for (i = 0; i < M; i++)
{
    for (j = 0; j < P; j++)
        printf("%d ", C[i][j]);

    printf("\n");
}

return 0;
}

```

Output:-



```

0 11 22
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Execution Time: 0.007558 seconds
Result Matrix C:
0 5 10
0 8 16
0 11 22

```

3. Write an OpenMP program to compute the dot product of two vectors using the reduction

clause to sum up partial results.

=>

```

#include <stdio.h>
#include <omp.h>

```

```

#define SIZE 100000

```

```

int main()
{
    double A[SIZE], B[SIZE];
    double sum = 0.0;
    int i;

```

```

    /* Initialize arrays */
    for (i = 0; i < SIZE; i++)
    {
        A[i] = 1.0;
        B[i] = 2.0;
    }

```

```

    double start_time = omp_get_wtime();

```

```

    #pragma omp parallel for reduction(+:sum)

```

```

for (i = 0; i < SIZE; i++)
{
    sum += A[i] * B[i];
}

double end_time = omp_get_wtime();

printf("Dot Product = %lf\n", sum);
printf("Execution Time = %f seconds\n", end_time - start_time);

return 0;
}

```

Output:-

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Dot Product = 200000.000000
Execution Time = 0.006565 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ █

```

4. Write an OpenMP program to demonstrate the difference between static and dynamic scheduling in OpenMP by summing elements of a large array.

=>

```

#include <stdio.h>
#include <omp.h>

```

```

#define SIZE 100000

```

```

int main()
{

```

```

    static double arr[SIZE];
    double sum_static = 0.0, sum_dynamic = 0.0;
    double start_time, end_time;
    int i;

```

```

    /* Initialize array */
    for (i = 0; i < SIZE; i++)
    {
        arr[i] = 1.0;
    }

```

```

    omp_set_num_threads(4);

```

```

    // ===== STATIC SCHEDULING =====
    start_time = omp_get_wtime();

```

```

    #pragma omp parallel for reduction(+:sum_static) schedule(static)
    for (i = 0; i < SIZE; i++)

```

```

{
    if (i % 1000 == 0)
    {
        for (int j = 0; j < 1000; j++);
    }

    sum_static += arr[i];
}

end_time = omp_get_wtime();

printf("STATIC Scheduling:\n");
printf("Sum = %lf\n", sum_static);
printf("Execution Time = %f seconds\n\n", end_time - start_time);

// ===== DYNAMIC SCHEDULING =====
start_time = omp_get_wtime();

#pragma omp parallel for reduction(+:sum_dynamic) schedule(dynamic, 1000)
for (i = 0; i < SIZE; i++)
{
    if (i % 1000 == 0)
    {
        for (int j = 0; j < 1000; j++);
    }

    sum_dynamic += arr[i];
}

end_time = omp_get_wtime();

printf("DYNAMIC Scheduling:\n");
printf("Sum = %lf\n", sum_dynamic);
printf("Execution Time = %f seconds\n", end_time - start_time);

return 0;
}

```

Output:-

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
STATIC Scheduling:
Sum = 100000.000000
Execution Time = 0.000786 seconds

DYNAMIC Scheduling:
Sum = 100000.000000
Execution Time = 0.000306 seconds
```


Lab-12: OpenMP-IV-Synchronization, Data Sharing,and Bitonic Sorting using OpenMP

AIM: To study the core features of OpenMP and apply them to exercises.

- 1. Synchronization: Lock Routines**
- 2. Environment Variables**
- 3. Data Environment**
- 4. Data Sharing Clauses**

EXERCISES:

1. Write an OpenMP program using 4 threads, each thread calculates factorial of its id and then all the factorials respective to threads need to be added to get to the final sum. Use shared and private clauses.

Output: Individual threads factorial result with their respective ids and the final sum of all the factorials.

=>

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int factorial(int n)
```

```
{
```

```
    int i, fact = 1;
```

```
    for (i = 1; i <= n; i++)
```

```
        fact = fact * i;
```

```
    return fact;
```

```
}
```

```
int main()
```

```
{
```

```
    int tid;
```

```
    int sum = 0;
```

```
    #pragma omp parallel num_threads(7) private(tid) shared(sum)
```

```
{
```

```
    tid = omp_get_thread_num();
```

```
    int fact = factorial(tid);
```

```

printf("Thread ID = %d , Factorial = %d\n", tid, fact);

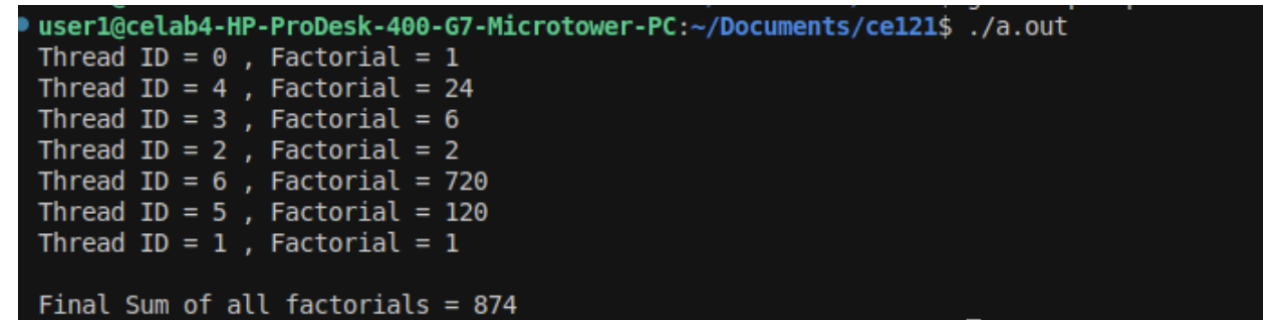
#pragma omp critical
{
    sum += fact;
}
}

printf("\nFinal Sum of all factorials = %d\n", sum);

return 0;
}

```

Output :



```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel121$ ./a.out
Thread ID = 0 , Factorial = 1
Thread ID = 4 , Factorial = 24
Thread ID = 3 , Factorial = 6
Thread ID = 2 , Factorial = 2
Thread ID = 6 , Factorial = 720
Thread ID = 5 , Factorial = 120
Thread ID = 1 , Factorial = 1
Final Sum of all factorials = 874

```

2. Write an OpenMP program for the Pi program by making minimum changes in the serial pi program that you can do.

=>

```

#include <stdio.h>
#include <omp.h>

static long num_steps = 100000;
double step;

int main()
{
    int i;
    double x, pi, sum = 0.0;

    step = 1.0 / (double) num_steps;

    #pragma omp parallel for private(x) reduction(+:sum)
    for (i = 0; i < num_steps; i++)

```

```

{
    x = (i + 0.5) * step;
    sum = sum + 4.0 / (1.0 + x * x);
}

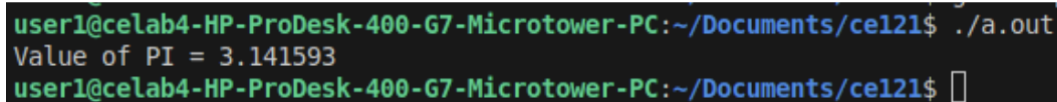
pi = step * sum;

printf("Value of PI = %lf\n", pi);

return 0;
}

```

Output :



```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ ./a.out
Value of PI = 3.141593
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/cel21$ 

```

3. Create a program that executes two independent tasks in parallel using OpenMP's sections directive.

=>

```

#include <stdio.h>
#include <omp.h>
#include <math.h>

```

```

/* ----- LASTPRIVATE EXAMPLE ----- */
void lastprivate_example(int n, double *lastterm)
{
    double x = 2.0;
    int i;

    #pragma omp parallel for lastprivate(x)
    for (i = 0; i < n; i++)
    {
        x = 9 + sqrt(x) * sqrt(x);

        printf("L : Iteration %d executed by thread %d, x = %f\n",
            i, omp_get_thread_num(), x);
    }

    *lastterm = x; // value from last iteration
}

```

```

/* ----- FIRSTPRIVATE EXAMPLE ----- */
void firstprivate_example()
{
    int a = 10;

    #pragma omp parallel firstprivate(a)
    {
        a = a + omp_get_thread_num();

        printf("F : Thread %d has value of a = %d\n",
            omp_get_thread_num(), a);
    }

    printf("F : Original value of a in main thread = %d\n", a);
}

```

```

/* ----- PRIVATE EXAMPLE ----- */
void private_example()
{
    int i;

    #pragma omp parallel for private(i)
    for (i = 0; i < 5; i++)
    {
        printf("P : Thread %d working on i = %d\n",
            omp_get_thread_num(), i);
    }
}

```

```

int main()
{
    double result;

    #pragma omp parallel sections
    {
        #pragma omp section
        {

```

```

        printf("\n--- FIRSTPRIVATE Example ---\n");
        firstprivate_example();
    }

#pragma omp section
    {
        printf("\n--- PRIVATE Example ---\n");
        private_example();
    }

#pragma omp section
    {
        printf("\n--- LASTPRIVATE Example ---\n");
        lastprivate_example(5, &result);
        printf("L : Last term = %f\n", result);
    }
}

return 0;
}

```

Output :

```

user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/ce121$ ./a.out
P : Thread 0 working on i = 1
P : Thread 0 working on i = 2
P : Thread 0 working on i = 3
P : Thread 0 working on i = 4

--- FIRSTPRIVATE Example ---

--- LASTPRIVATE Example ---
F : Thread 0 has value of a = 10
F : Original value of a in main thread = 10
L : Iteration 0 executed by thread 0, x = 9.000000
L : Iteration 1 executed by thread 0, x = 18.000000
L : Iteration 2 executed by thread 0, x = 27.000000
L : Iteration 3 executed by thread 0, x = 36.000000
L : Iteration 4 executed by thread 0, x = 45.000000
L : Last term = 45.000000

```

4. Study and Implement Bitonic Sorting in OpenMP. Use the other PDF attached with this for implementation details.

=>

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

```

```

#include <time.h>

#define ASCENDING 1
#define DESCENDING 0

int is_power_of_two(int n)
{
    return (n > 0) && ((n & (n - 1)) == 0);
}

void compare_exchange(int *a, int *b, int dir)
{
    if ((dir == ASCENDING && *a > *b) ||
        (dir == DESCENDING && *a < *b))
    {
        int temp = *a;
        *a = *b;
        *b = temp;
    }
}

void bitonic_merge(int arr[], int low, int cnt, int dir)
{
    if (cnt > 1)
    {
        int k = cnt / 2;

        #pragma omp parallel for schedule(static)
        for (int i = low; i < low + k; i++)
        {
            compare_exchange(&arr[i], &arr[i + k], dir);
        }

        bitonic_merge(arr, low, k, dir);
        bitonic_merge(arr, low + k, k, dir);
    }
}

void bitonic_sort(int arr[], int low, int cnt, int dir)
{

```

```

if (cnt > 1)
{
    int k = cnt / 2;

    #pragma omp task shared(arr) if(cnt >= 2048)
    bitonic_sort(arr, low, k, ASCENDING);

    #pragma omp task shared(arr) if(cnt >= 2048)
    bitonic_sort(arr, low + k, k, DESCENDING);

    #pragma omp taskwait

    bitonic_merge(arr, low, cnt, dir);
}
}

void parallel_bitonic_sort(int arr[], int n, int dir)
{
    #pragma omp parallel
    {
        #pragma omp single
        {
            bitonic_sort(arr, 0, n, dir);
        }
    }
}

int is_sorted(int arr[], int n, int dir)
{
    for (int i = 0; i < n - 1; i++)
    {
        if (dir == ASCENDING)
        {
            if (arr[i] > arr[i + 1])
                return 0;
        }
        else
        {
            if (arr[i] < arr[i + 1])
                return 0;
        }
    }
}

```

```

    }
}
return 1;
}

int main()
{
    int n;

    printf("Enter number of elements (power of 2): ");
    scanf("%d", &n);

    if (!is_power_of_two(n))
    {
        printf("Error: n must be a power of 2.\n");
        return 1;
    }

    int *arr = (int *)malloc(n * sizeof(int));

    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    srand((unsigned int)time(NULL));

    for (int i = 0; i < n; i++)
    {
        arr[i] = rand() % 1000;
    }

    printf("\nOriginal array will be Random.\n");

    int ARRforASC[n], ARRforDES[n];

    for (int i = 0; i < n; i++)
    {
        ARRforASC[i] = ARRforDES[i] = arr[i];
    }
}

```



```

}

// ASCENDING
double start = omp_get_wtime();
parallel_bitonic_sort(ARRforASC, n, ASCENDING);
double end = omp_get_wtime();

printf("\nSorted array : Ascending : ");
printf("\nSorted correctly? %s\n",
    is_sorted(ARRforASC, n, ASCENDING) ? "Yes" : "No");

printf("Execution time = %.6f seconds\n", end - start);

// DESCENDING
start = omp_get_wtime();
parallel_bitonic_sort(ARRforDES, n, DESCENDING);
end = omp_get_wtime();

printf("\nSorted array : Descending : ");
printf("\nSorted correctly? %s\n",
    is_sorted(ARRforDES, n, DESCENDING) ? "Yes" : "No");

printf("Execution time = %.6f seconds\n", end - start);

free(arr);
return 0;
}

```

Output :

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/ce121$ ./a.out
Enter number of elements (power of 2): 262144

Original array will be Random.

Sorted array : Ascending :
Sorted correctly? Yes
Execution time = 0.562816 seconds

Sorted array : Descening :
Sorted correctly? Yes
Execution time = 0.554181 seconds
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/ce121$ ^C
```

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/ce121$ export OMP_NUM_THREADS=64
./a.out
Enter number of elements (power of 2): 262144

Original array will be Random.

Sorted array : Ascending :
Sorted correctly? Yes
Execution time = 0.494442 seconds

Sorted array : Descening :
Sorted correctly? Yes
Execution time = 0.452527 seconds
```

```
user1@celab4-HP-ProDesk-400-G7-Microtower-PC:~/Documents/ce121$ export OMP_NUM_THREADS=2
./a.out
Enter number of elements (power of 2): 262144

Original array will be Random.

Sorted array : Ascending :
Sorted correctly? Yes
Execution time = 1.061005 seconds

Sorted array : Descening :
Sorted correctly? Yes
Execution time = 1.050188 seconds
```